

Constrain the Target, Not the Loss: Action-Space Trust Regions for Critic-Guided Policy Fine-Tuning

Anonymous authors
Paper under double-blind review

Abstract

Behavior cloning has made visuomotor robot policies easy to train, but their performance is bounded by the demonstrations they imitate. Reinforcement learning offers a way past that bound by letting the policy practice against a learned critic, yet in the regime that matters in practice — a policy cloned from few demonstrations, improved online, rewarded only when the task succeeds — critic-guided fine-tuning is unreliable and frequently destroys the policy it was meant to improve. The cause is that a critic is accurate only on the narrow set of actions it was trained on, while the standard update lets that critic determine both the direction and the magnitude of every policy change. The policy is therefore pushed outside the region the critic understands, where it stops succeeding and, under sparse reward, receives no signal that would guide it back.

We present a fine-tuning rule that separates these two roles: the critic chooses the direction of the update, and the algorithm chooses the distance. Candidate actions are drawn from the frozen pretrained policy, displaced along the critic’s gradient, pulled back whenever they stray beyond a fixed radius of the action they started from, and regressed into a lightweight residual head. The critic, replay data and interaction budget are unchanged from the strongest published residual fine-tuner, so the actor update is the only variable.

Across two benchmarks this yields +9.4 points on the eight hardest LIBERO-90 tasks where backpropagating the critic yields +0.9, and raises robomimic square from 0.382 to 0.91–0.93, while published methods that let the critic set the distance fail to improve their base and in two cases collapse to zero. Ablating our own components isolates why: bounding how far the policy moves per step changes nothing, and the component that matters is the one holding the policy near where it started. The effective constraint is a leash to a fixed anchor, not a speed limit.

1 Introduction

Behavior cloning has become the default way to build visuomotor robot policies, and it works: given enough demonstrations, a cloned policy performs the task it was shown. Its ceiling, however, is the demonstrations themselves. Teleoperated data is expensive to collect, human operators are rarely optimal, and there is mounting evidence that policy performance saturates as demonstrations accumulate. Getting a deployed policy past that ceiling is now one of the practical bottlenecks in robot learning.

Reinforcement learning is the natural complement: let the policy practice in its environment and improve from its own experience. Applying it directly to a modern generative policy is awkward, though, because such policies map noise to multimodal action chunks and expose no tractable likelihood, which rules out the PPO-style machinery that most of RL is built on. The workarounds either inject noise into the deployed policy (Zhang et al., 2026), changing what ships, or assume a denoising objective the policy does not have (McAllister et al., 2025).

This is why the dominant practical recipe is residual RL: freeze the pretrained policy, learn a small corrective term on top of it, and score candidate actions with a learned critic (Sun & Song, 2026). The base supplies competence and the critic supplies improvement.

That recipe has a failure mode we believe is underappreciated, and it is specific to the setting practitioners care about most: a policy cloned from few demonstrations, improved online, with reward that fires only on task success. In that regime, critic-guided fine-tuning frequently makes the policy worse, sometimes catastrophically so. Running three published action-gradient fine-tuners from their authors’ own code, at $125\times$ our interaction budget, none improves its starting policy and two collapse to exactly 0.000 success (Table 1).

The cause is a division of labour nobody chose deliberately. A critic is trained to rank actions in the region its data covers, and it does that well. Nothing constrains it outside that region, where it can be confidently and steeply wrong (Fujimoto et al., 2018; 2019; Jain et al., 2025). Yet the standard update lets the critic decide not only which way the policy should change but — through the magnitude of its gradient — how far. Only the first is a question the critic was trained to answer. The consequences are not symmetric: a slightly wrong direction gives a slightly wrong step, whereas a wrongly large distance carries the policy out of the region the critic understands, and the further out it goes the better it looks to the critic that sent it there.

Figure 1 shows this with nothing hidden. We fit a critic to 25 demonstrations lying on a narrow band of workable actions and follow its gradient from a demonstrated action. The policy walks off the band to a point the critic scores 1.63 — higher than any reward the task can return — where the true reward is exactly 0.000. Repeat with 700 demonstrations and the effect vanishes: the critic has seen enough to know the band. The failure is not a property of critics in general but of post-training from few demonstrations. Sparse reward is what makes it terminal rather than temporary: under dense reward a policy that wanders still receives graded feedback and can recover, and we verify that the same unconstrained update survives there; under success-only reward, a policy that has stopped succeeding sees nothing but zeros and has no signal with which to return.

We propose to split the two roles apart, in a method we call CAST (Constrained Action-Space Transport). The critic keeps the job it is good at — choosing the direction in which each action should change — and the algorithm takes the job the critic was never trained for: deciding how far the policy may end up from the policy we started with. Concretely, candidate actions are drawn from the frozen base, displaced along the critic’s gradient, pulled back whenever they stray beyond a fixed radius of their starting action, and regressed into a lightweight residual head. Nothing else in the pipeline changes, so the actor update is the only variable in every comparison we report.

Our contributions:

- A diagnosis of why critic-guided post-training fails from few demonstrations (Section 3.3, Figure 1). We show, on a fitted critic rather than by argument, that the failure is an extrapolation problem with a sharp data threshold: at 25 demonstrations the critic reports a value above anything the task can return at an action whose true reward is zero, and at 700 the effect disappears. We further show that sparse reward is what makes the failure permanent, by exhibiting the same unconstrained update surviving under dense reward.
- An actor update that separates direction from distance (Section 4). Improving actions along $\nabla_a Q$ and regressing the policy onto them is an established family (Li et al., 2024; Psenka et al., 2025; Choi et al., 2026); we adopt that form and add what it lacks — a bound, placed in action space where the harm occurs rather than in the loss, where a penalty must out-shout a gradient of unbounded size and loses (0.549 and 0.601 against bases of 0.573 and 0.610). Two further choices separate our instantiation: targets come from freshly drawn particles off a frozen base rather than buffer actions mutated in place, so stored transitions remain consistent with the dynamics that produced them, and the update needs no likelihood.
- Evidence that the update rule is what wins, under a strict control (Section 5). Holding base, critic, data and budget fixed and varying only the actor update, our

rule gains +9.4 points on the eight hardest LIBERO-90 tasks where backpropagating $-Q$ gains +0.9, lifts robomimic square from 0.382 to 0.91–0.93, and transfers unchanged across generator classes. Two controls close the obvious alternative explanations: sweeping each method’s step size an order of magnitude leaves the two in non-overlapping bands, so the gap is not tuning; and deleting only the critic’s gradient direction collapses performance below base, so the gain is not mere caution.

- An ablation that overturns the intuitive explanation (Section 5.4). The natural reading of our own method is that it works by limiting step size. It does not. With the step clip alone success is exactly 0.000 — identical to using no constraint — because a drift assembled from many individually permitted steps arrives just as surely as one large one. What makes the update viable is the term anchored to the frozen base. The effective constraint is a tether to a fixed reference, not a speed limit, and we report this because it contradicts what we expected to find.

2 Related Work

RL fine-tuning of generative robot policies. Policy-gradient fine-tuning of vision-language-action models with sparse task reward is now standard for autoregressive token policies (Li et al., 2025), but transfers poorly to continuous generative heads: likelihood ratios are intractable for diffusion and flow policies, and workarounds either reformulate the denoising chain as an MDP (Ren et al., 2025; Black et al., 2024) — a construction with no analog for one-step generators — or avoid likelihoods entirely (Park et al., 2025; Zhang et al., 2026; Mark et al., 2024). FPO (McAllister et al., 2025) is the strongest form of the likelihood-free ratio: it replaces $\pi_\theta/\pi_{\theta_{\text{old}}}$ with the exponentiated difference of per-sample conditional flow-matching losses, which the CFM–ELBO relationship makes a valid proxy, and it does so without perturbing the deployed policy. It is therefore the right policy-gradient method for a flow or diffusion policy — but the surrogate is defined by a denoising loss over a flow time τ , and a one-step drifting model has neither: its training objective is field-target regression, with no ELBO to bound. FPO also fine-tunes (or trains from scratch) the full policy on continuous-control suites, where we update a residual on a frozen base for manipulation. The two are complementary — FPO makes the ratio tractable, we make the critic safe to follow — and we regard evaluating FPO on a flow-matching base against our update as the clearest missing comparison in this paper. Closest to our setting, DICE-RL (Sun & Song, 2026) fine-tunes a frozen one-step distillation of a flow-matching policy by training a residual head with a backpropagated critic and a filtered BC anchor; it is our primary baseline and the control for our actor-update ablation, since we keep its critic, residual parameterization, and data pipeline unchanged.

Where this update sits. RL for diffusion and flow policies divides into four families (Choi et al., 2026): backpropagating through the denoising chain (Diffusion-QL, and DICE-RL’s residual actor); ratio-based methods that make a likelihood tractable (DPPO, FPO); Q -weighted regression; and action-gradient methods, which improve actions with $\nabla_a Q$ and then regress the policy onto the improved actions. CAST belongs to the last family, and we want to be exact about what is and is not new in it. The primitive of displacing actions along $\nabla_a Q$ and regressing onto the result is not ours: DIPO (Li et al., 2024) introduced it, updating replay-buffer actions by gradient ascent and fitting a diffusion policy to them, and QSM (Psenka et al., 2025) reaches the same signal differently, by matching the policy’s score to $\nabla_a Q$. What the family lacks — and what the survey notes none of its members supply — is any bound on how far that displacement may carry the policy, or any restoring force toward a behavior policy. That gap is also its documented weakness: action-gradient methods are reported to degrade when the critic’s gradient is biased, and to drift away from the behavior distribution.

Our contribution is precisely that missing piece, and our ablations measure it directly. The displacement is clipped pointwise in action space, a dead-zone anchor restores toward a frozen base, and the update is applied to a residual rather than by overwriting stored transitions (DIPO’s buffer rewrite makes the stored action inconsistent with the transition that produced it). Two measurements tie our system to theirs. Removing the base-restoring

anchor reproduces the family’s failure inside our own code exactly — success 0.000, whether or not the pointwise clip is kept (Table 4) — so we can switch their failure on and off. And running DIPO, QSM and DQL from their authors’ implementations on our setting, at $125\times$ our interaction budget, none improves its base and two reach exactly 0.000 (Table 1), which is the same failure observed from the outside. We are careful about the direction of this claim: these methods were developed for dense-reward online control, and the adaptation to sparse-reward manipulation fine-tuning is the one published by Ren et al. (2025), not one we devised. Their difficulty here is evidence about the regime, not a verdict on the methods. Test-time guidance (Zhou et al., 2026) applies the same $\nabla_a Q$ signal without training at all, and is complementary.

Critic-as-compass versus critic-as-judge. That a learned Q -function’s action-gradient cannot be trusted is a recurring diagnosis: overestimation and narrow spurious peaks in TD3 (Fujimoto et al., 2018), extrapolation error in BCQ (Fujimoto et al., 2019), and spurious local optima that make sample-and-select beat gradient ascent (Jain et al., 2025). One response treats the critic only as a judge: weighted regression and sample-based improvement (AWR, CRR, MPO, IDQL, QT-Opt) (Peng et al., 2019; Wang et al., 2020; Abdolmaleki et al., 2018; Hansen-Estruch et al., 2023; Kalashnikov et al., 2018) tilt the policy toward $\pi \cdot \exp(Q/\tau)$ using only Q -values. Our method spans this spectrum rather than picking a side: the update form is judge-safe (bounded transport toward targets), while the reward field inside it can still be the analytic gradient where it is trustworthy — and our ablations map exactly when each choice wins. The judge-only methods are also the ones that survive our benchmark: of the published fine-tuners we ran, the one that only weights actions rather than displacing them is the only one to improve its base (Table 1). It pays for that safety by discarding the critic’s most informative output, which is what our bounded update is designed to keep.

Particle transport as a policy update. Regressing a sampler onto displaced particles is the amortized-SVGd idea of Soft Q -Learning (Haarnoja et al., 2017), with kernel repulsion preserving diversity as in SVPG (Liu et al., 2017); drifting models (Deng et al., 2026) make the same operator the pretraining objective of a one-step generator. Drifting Field Policy (Koo et al., 2026) is the closest prior method: it also performs RL on a drifting backbone by drifting fresh policy samples toward critic-ranked top- K candidates. CAST differs in three ways: we update a residual on a frozen base rather than the full policy, we transport with an explicit pointwise trust region and a restoring anchor field (stability contract our toy shows is load-bearing), and our reward field is a dial — including the analytic $\nabla_a Q$ source that dominates when the critic generalizes — rather than a fixed ranking surrogate.

Residual RL and staying on-manifold. Learning a residual on a frozen base is a long-standing recipe for sample-efficient robot RL; DAWN (Ma et al., 2026) identifies the value cold-start and scale pathologies of residual critics that we also observe. The off-manifold drift our toy exhibits is the mechanism behind over-optimization and mode collapse in RL-finetuned generators (Black et al., 2024; Fan et al., 2023; Nakamoto et al., 2023), and forgetting in RL-finetuned language models is likewise governed by divergence from the base (Shenfeld et al., 2025) — which cannot even be measured without likelihoods, reinforcing update rules whose step size is bounded in action space by construction.

In sum, prior work either trusts the critic’s gradient and pays for it off-manifold, or discards it and pays on hard tasks; CAST is, to our knowledge, the first fine-tuner for one-step generative policies whose update form is safe under an arbitrary critic while still admitting the full first-order signal when it is available.

3 Problem Statement

3.1 The setting: post-training a cloned policy

The situation this paper addresses is concrete. A robot policy has been trained by behavior cloning on a modest number of demonstrations — tens, not thousands — and it works,

imperfectly. We now want it to get better by practising in its environment, and the only feedback available is whether the episode succeeded. Three features of this setting do the work in everything that follows, and each is a property of practice rather than a modelling choice:

- The demonstrations are few. Whatever a learned critic knows, it learned from a thin shell of actions around them.
- The reward is sparse. A policy that stops succeeding receives a stream of zeros, which carries no information about how to succeed again.
- The base policy is worth protecting. It is the only thing in the system that reliably does the task at all.

Formally, states are s , actions are chunks $a \in \mathbb{R}^{H \times d}$ of H control steps flattened to dimension $S = Hd$, and reward is a sparse task-success signal. The pretrained policy maps a state and a noise sample to an action in a single network evaluation,

$$a = \pi_0(s, z), \quad z \sim \mathcal{N}(0, I), \quad (1)$$

so the policy is a deterministic map from noise to action. Following standard residual fine-tuning practice (Ma et al., 2026; Sun & Song, 2026), we freeze it and learn a small residual on top,

$$a = \pi_0(s, z) + r_\theta(s, z), \quad (2)$$

alongside an ensemble critic $Q_\phi(s, z, a)$ trained by temporal difference with a conservative penalty (Kumar et al., 2020). Critic training is identical for every method we compare. The object of study is only the actor update — how the policy consumes the critic it is given.

3.2 Why the obvious tools do not apply

Policy gradients need a likelihood this policy does not have. PPO- and GRPO-style finetuners (Li et al., 2025) require $\log \pi(a | s)$. A one-step generator defines its action distribution implicitly through the noise map, its likelihood is intractable, and unlike multi-step diffusion there is no chain-MDP reformulation to fall back on (Ren et al., 2025; Park et al., 2025). Two routes recover a ratio and neither is free. One changes the deployed policy, adding Gaussian noise to its output so a surrogate density exists (Zhang et al., 2026); we include it as a baseline and it is competitive, but what ships is no longer the policy that was pretrained. The other leaves the policy alone but requires it to be a flow model: FPO (McAllister et al., 2025) builds a clip ratio from differences of conditional flow-matching losses, which needs a denoising loss over a flow time τ that a one-step model does not have. So a critic is what remains, and the question becomes how to consume it.

Every critic-based update hands the critic two jobs. A critic can answer one question well: given this action, which way should it change to score higher. Standard updates ask it a second question at the same time — how far — and take the answer from the magnitude of its gradient. Backpropagating $-Q$ through the actor, as in the DDPG lineage and DICE-RL’s residual actor (Sun & Song, 2026), does this explicitly,

$$\nabla_\theta \mathbb{E}[-Q] = -\nabla_a Q \cdot \partial a / \partial \theta, \quad (3)$$

so a steeper critic produces a larger parameter step. The action-gradient family (Li et al., 2024; Psenka et al., 2025) never differentiates through the policy, but the improved action lands wherever $a + \alpha \nabla_a Q$ falls, so the displacement is again proportional to the critic’s steepness — the same delegation, one level up.

Only the first question is one the critic was trained to answer. It learns to rank actions where it has data; away from that data nothing constrains it, and it can be steep and wrong at once (Fujimoto et al., 2018; 2019; Jain et al., 2025). The two errors then fail asymmetrically. A wrong direction costs a slightly wrong step. A wrong distance costs the data manifold — and once the policy is off it, higher critic scores mean worse behavior, because the critic is scoring a region it has never been corrected on.



Figure 1: Why critic-guided post-training leaves the data, and what a bound does about it. Panels (a)–(c) are computed, not drawn: a critic is fit by regression to 25 demonstrations, the number a post-training setting actually has. (a) The task’s true reward, and the demonstrations — all of which lie on one narrow band of feasible actions. (b) What the critic learned. On the band it is accurate; beyond the dashed contour it has no data and is extrapolating, and nothing in its training penalises it for being confidently wrong there. (c) Two updates from the same start, both following $\nabla_a \hat{Q}$. The unconstrained one walks off the band to a point its own critic scores $\hat{Q} = 1.63$ — higher than any achievable reward — where the true reward is exactly 0.000. Bounding the displacement around the frozen base keeps the policy where the critic is valid and improves it ($0.016 \rightarrow 0.040$, $2.5\times$). The gap between what the critic claims and what the policy earns is the whole failure. (d) The same mechanism measured in the real system: distance from the frozen base during LIBERO the red-mug task training. Without a base-restoring term the policy walks away without bound and success reaches 0.000; with it the policy sits at the trust radius for the entire run. The regime matters: repeating (a)–(c) with 700 demonstrations instead of 25 removes the effect entirely (critic then learns the band and its gradient is trustworthy), which is why this paper scopes its claims to post-training from few demonstrations.

One detail decides which remedy can work, so we state it early: the damage is persistent, not sudden. The critic’s extrapolated optimum does not recede as the policy approaches it, so every update points the same way and nothing pulls the policy back. A limit on the size of each step therefore does not prevent the drift — it only slows it — while a force that returns the policy toward its starting point does. Section 5.4 measures both and confirms the distinction.

3.3 Watching it happen

The argument above is easy to state and easy to doubt, so we make it visible in a setting small enough to draw completely and honest enough to be recomputed: two-dimensional actions, a curved band of workable actions standing in for the data manifold, and a reward that decays away from that band. We sample 25 demonstrations from the band — the number a post-training run actually has — and fit a critic to them by regression. Nothing about the critic is rigged: it sees only demonstrated actions and their true rewards, exactly as a critic trained on demonstration data would.

Three things follow, and all are computed rather than asserted.

The critic is accurate where the data is, and invents elsewhere. Because every demonstration lies on the band, nothing in the critic’s training tells it that leaving the band is bad. Its predictions there are extrapolation, and Figure 1(b) marks the contour beyond which they stop matching reality.

Following its gradient walks the policy off the band. Starting from a demonstrated action and taking repeated steps along $\nabla_a \hat{Q}$ lands at a point the critic scores $\hat{Q} = 1.63$ — higher than any reward the task can produce — where the true reward is exactly 0.000. The



Figure 2: The two-regime toy (3 seeds). Left, easy regime (2-D actions, dense critic data): the critic’s gradient generalizes and backpropagation ($-Q$, with or without a BC anchor) is the best update. Right, hard regime (16-D actions, 60 critic points): the same updates collapse to zero true reward while their internal Q -estimate explodes — the BC anchor does not help — whereas the bounded field-target update stays on-manifold and holds the base policy. A useful fine-tuner must keep the gradient where it is trustworthy and survive where it is not. This sweeps the dimension of the action space at fixed data, and Figure 1 sweeps the amount of data at fixed dimension; both cross the same boundary, which is what the critic’s coverage of the region the update moves into.

policy has not become confused; it has done exactly what it was told, by a critic that was confidently wrong. Bounding the displacement around the frozen starting action instead keeps the policy where the critic is informative and improves it ($0.016 \rightarrow 0.040$).

The effect has a data threshold, and that threshold defines this paper’s scope. Repeating the identical construction with more demonstrations removes it: at 200 demonstrations the critic’s endpoint earns a true reward of 0.901 against its own estimate of 0.910, and at 700 the two agree just as closely. With enough data the critic learns the band and its gradient is worth following without any bound at all. We therefore do not claim that critic gradients are untrustworthy in general — only that they are untrustworthy in the regime where post-training actually happens, and we scope every claim in this paper accordingly.

The same boundary can be crossed a second way, which rules out the objection that our two-dimensional picture is a special case. Holding the amount of data fixed and raising the dimension of the action space shrinks the fraction of that space the critic has seen, and produces the identical two regimes (Figure 2): with 2-D actions the critic’s gradient generalizes and backpropagation is the best available update, while with 16-D actions and the same critic budget the same update drives true reward to zero as its internal Q -estimate explodes. Coverage is the variable in both experiments; dimension and demonstration count are two ways of changing it.

4 Method

Section 3.3 left us with a concrete requirement: follow the critic where it has data, and prevent the policy from wandering to where it does not. Crucially, this is a requirement about where the policy ends up, not about how fast it travels — a distinction the experiments turn out to hinge on.

Meeting it requires answering one question: where does the critic stop being reliable, and how do we express that boundary in a form an update can respect? Our answer is deliberately blunt. The boundary is not something we can compute — if we could estimate the critic’s competence region we would already have solved the problem — so we approximate it by the only landmark available: the frozen policy that the critic was trained around. Actions near its output are actions the critic has seen; actions far from it are not. The constraint therefore takes the form of a radius about the base policy’s own action, and the method is what follows from taking that seriously.

The vehicle is an existing primitive, which we adopt rather than claim: move action samples along a velocity field, then regress the network onto the moved samples. This is how the action-gradient family improves a policy (Li et al., 2024; Psenka et al., 2025). What that family does not do, and what this section contributes, is constrain the displacement: in DIPO the improved action is wherever the gradient step lands, and the policy is regressed onto it, wherever that is.



Figure 3: CAST in three steps (schematic). Left: the frozen base maps noise to K candidate actions, which lie on the thin band of actions the demonstrations support. Middle: the critic’s gradient points off that band (faint arrows, unbounded); CAST keeps only its direction and takes a step of length at most δ_Q (solid), then the dead-zone anchor pulls back any candidate that has left the leash of radius ρ (green). Right: the displaced targets are detached and the residual head is regressed onto them, so the policy moves a bounded distance and stops. The base is any frozen noise-to-action map.

Nothing below is specific to a policy class. The update needs a frozen map from noise to actions, the ability to draw K samples from it, and a critic. It never needs a likelihood, a denoising objective, or access to the generator’s internals, so it applies unchanged to a one-step map, a few-step flow-matching sampler, or a many-step diffusion policy, and we verify this empirically across generator classes (Section 5.7).

The contribution is what happens between the critic and the regression: the proposed displacement is constrained in action space, on the target, before anything is fitted. The critic proposes an improvement; an explicit action-space trust region limits how much of that proposal the actor is asked to learn; only the constrained target is regressed. This is the difference from the backpropagated control, which admits the same proposal into a loss and lets it compete there against a regularizer (Section 4.4).

We instantiate the constraint two ways and claim neither over the other: a pointwise clip on each particle’s displacement, and an anchor field V_{anc} — a pull back toward the frozen base that is silent while the residual stays within a radius ρ and engages only outside it (Section 4.3 gives its form). The anchor is not an ad hoc construction: it is the negative gradient of a hinged radial potential, and at $\rho = 0$ it reduces exactly to the quadratic residual regularizer the backpropagated control already uses. The dead zone is thus a hinge generalization of standard L_2 residual regularization — which makes ρ an explicit interpolation between the loss-space regularizer of prior work and the constraint we advocate, and lets us measure the two against each other inside one implementation (Section 5.5).

The reward field V_Q says which direction improves each action, and its magnitude enters the target before either bound acts. We do not attribute the method’s behaviour to any single one of these components: what the experiments establish is that some action-space bound must be present (without one, success is exactly 0.000), and that within a wide band its particular form is not distinguishable at this budget (Section 5.4).

Neither field is a loss term; both are displacements applied to actions before the regression. This does not make the anchor invincible — where $\|V_Q\| > \|V_{\text{anc}}\|$ the net displacement still points outward — but it means the two compete as bounded magnitudes in action space rather than through an unbounded parameter gradient, as a behaviour-cloning penalty must (Section 3.3). No likelihood is required, and no critic gradient is ever backpropagated into the actor.

4.1 CAST in four steps

We call the method CAST — Constrained Action-Space Transport — because that is literally what it does: it transports candidate actions toward higher critic value, and it constrains that transport in action space, where the damage occurs, rather than in the loss or in parameter space.

Before any notation, here is the whole method in one picture. For a given state, the frozen policy can produce many plausible actions — draw different noise, get different candidates. We take a handful of them and ask the critic to improve each one, but we treat its answer as advice about direction and cap how far we act on it. Any candidate that ends up too far from the action the frozen policy originally proposed is pulled back toward it. Then we simply train the residual head to reproduce the corrected candidates, as if they were demonstrations. The critic is never differentiated through; it is only ever asked a question and answered.

So one update does four things:

1. Propose. Draw K noise samples and push them through the frozen base plus the current residual, giving K candidate actions for this state.
2. Nudge. Ask the critic which direction improves each candidate, and take a step of bounded length in that direction — the critic sets the heading, we set the stride.
3. Leash. If a candidate has drifted more than ρ away from what the frozen policy proposed, pull it back toward that action. Inside ρ nothing happens, so the critic has free rein where it is trustworthy.
4. Imitate. Detach the corrected candidates and train the residual head by regression onto them. They are data now, not a computation graph.

Two consequences are worth noticing immediately. Because step 4 is plain regression onto fixed targets, the size of the parameter update is set by us, not by how steep the critic happens to be. And because step 3 measures distance from the frozen policy rather than from the previous iterate, the reference point never moves — which, as Section 5.4 shows, is exactly what separates a constraint that works from one that does not.

Concretely: for each state in the batch we draw K noise samples and form the current action candidates $a_i = \pi_0(s, z_i) + r_\theta(s, z_i)$, treated as constants (detached). Each is displaced by a combined velocity,

$$\tilde{r}_i = \text{sg}\left[r_i + \text{clip}_{\delta_Q}(V_Q(s, z_i, a_i)) + V_{\text{anc}}(r_i)\right], \quad \mathcal{L}_{\text{actor}} = \frac{1}{K} \sum_i \|r_\theta(s, z_i) - \tilde{r}_i\|^2, \quad (4)$$

where $\text{sg}[\cdot]$ is stop-gradient and $\text{clip}_\delta(v) = v \cdot \min(1, \delta/\|v\|)$ rescales a field vector to norm at most δ .

Matching this to the four steps: V_Q is the critic’s direction (step 2), clipped to length δ_Q ; V_{anc} is the leash (step 3), silent while $\|r_i\|$ stays within ρ ; the total displacement is clipped to δ_{tot} ; and $\text{sg}[\cdot]$ makes the result a fixed regression target (step 4). The critic is trained separately (Section 4.4).

Why displace targets rather than differentiate. Writing the update as regression onto displaced targets is what makes the boundary enforceable at all. The quantity we need to bound — how far the policy’s action ends up from the base policy’s action — is a property of the target, and here it is available explicitly, before any gradient is taken: we can inspect each displaced target, compare it to the action it came from, and correct it. Backpropagating $-Q$ offers no such handle, because the distance travelled is an emergent consequence of a parameter step and is not represented anywhere the algorithm can act on. Two secondary properties follow from the same choice. The regression target is detached, so the critic never differentiates through the actor and cannot inject unbounded curvature into the parameter update. And because targets are formed from freshly drawn samples rather than from stored actions rewritten in place, replayed transitions continue to describe the dynamics that produced them — a consistency DIPO’s buffer rewrite gives up (Li et al., 2024).

What the clip does and does not guarantee. The clip bounds target transport, and only that:

$$\|\tilde{r}_i - r_i\| \leq \delta_{\text{tot}} \quad \text{for every particle } i. \quad (5)$$

It is tempting to read this as a bound on how far the policy moves, and an earlier version of this work did. That inference is invalid. The targets are amortized through a shared network by a gradient step, so to first order the realized change at particle i is

$$\Delta r_i \approx \alpha \sum_j J_i J_j^\top (\tilde{r}_j - r_j), \quad J_i = \partial r_\theta(s, z_i) / \partial \theta, \quad (6)$$

and the Jacobian coupling across particles carries no guarantee that $\|\Delta r_i\| \leq \delta_{\text{tot}}$. This is precisely the objection we raise against parameter-space gradient clipping (Section 3.2), and it applies to target-space clipping too: bounding what the network is asked for does not bound what it does. A per-particle guarantee would require an output-space proximal step, exact interpolation, or a tabular actor — none of which we impose.

The distinction matters, but it does not undermine the construction, because a per-particle guarantee is not what the update needs. A drift assembled from many individually permitted steps is still a drift, since the critic keeps pointing the same way and nothing returns the policy — which is why the constraint that matters is one whose strength grows with the displacement already accumulated, rather than one that merely rate-limits each step. What we establish empirically is narrower and sufficient: with no action-space bound of any kind the update destroys the policy, and with either bound present it does not (Section 5.4).

4.2 The reward field V_Q

The default (and headline) reward field is the normalized critic gradient evaluated at each particle,

$$V_Q^{\text{grad}}(s, z, a) = \eta_Q \frac{\nabla_a \bar{Q}_\phi(s, z, a)}{q_{\text{scale}}}, \quad q_{\text{scale}} = \mathbb{E}[|\bar{Q}_\phi|], \quad (7)$$

where $\bar{Q}_\phi$ is the conservative ensemble aggregate. This is deliberately the same scalar the DICE-RL actor optimizes: in the small-step, unclipped limit, minimizing Equation (4) recovers the backpropagated update direction exactly (Proposition 1), so any performance difference is attributable to the update form — the anchored target regression — and not to a different reward signal.

The kernel transport operator. The lower-resolution reward fields below and the distributional anchor variant share one particle transport operator, which we take off the shelf (Deng et al., 2026); it is a convenience for turning a set of target particles into a velocity field, and nothing in the method depends on its particular form. Given query particles x , attractor particles p^+ (optionally weighted), and repulsor particles p^- , distances are normalized by a global scale and, for each kernel radius R in a small set, a soft mutual-matching affinity $A = \sqrt{\text{softmax}_{\text{row}}(-d/R) \odot \text{softmax}_{\text{col}}(-d/R)}$ converts matched attractors into pulls and matched repulsors into pushes; forces are summed over radii to give $\mathcal{V}(x \mid p^+, p^-)$ (full form in Appendix C). Intuitively, $\mathcal{V}$ moves a particle cloud so its density flows toward p^+ and away from p^- — it needs only samples of the target, never a density or a gradient.

Because V_Q enters Equation (4) only as a velocity, the critic can be consumed at lower resolution when its gradients are not trustworthy, using only Q -values:

- zeroth-order (top- k): rank the K particles by Q and set $V_Q = \mathcal{V}(a \mid \text{top-}k \text{ particles, all particles})$ — the pretraining kernel transport toward the critic’s preferred candidates;
- tilted: let every particle attract with weight $K \cdot \text{softmax}_i(\text{adv}_i/\tau)$, transporting the cloud toward its own $\exp(Q/\tau)$ -tilted density, the canonical improvement target of Peng et al. (2019); Abdolmaleki et al. (2018).

These are not competing methods but settings of a dial — how much of the critic’s local structure the update trusts: full first-order information, ranking only, or soft value weights.

Section 5 maps the regimes: the gradient source dominates wherever the critic generalizes, while the value-only sources close the gap precisely in the weak-critic regime predicted by Section 3.3.

4.3 The anchor field

The kernel reward fields vanish beyond their largest radius and the gradient field carries no notion of the data manifold, so Equation (4) alone would let the residual random-walk. The anchor is a restoring field with a dead zone,

$$V_{\text{anc}}(r) = -\eta_{\text{anc}} \left(1 - \frac{\rho}{\|r\|}\right)_+ r, \quad (8)$$

which is inert inside the trust radius ρ and pulls back only the excess residual norm beyond it. Equivalently, Equation (8) is the negative gradient of a hinged radial potential,

$$U(r) = \frac{1}{2} \eta_{\text{anc}} (\|r\| - \rho)_+^2, \quad V_{\text{anc}}(r) = -\nabla_r U(r), \quad (9)$$

so it removes exactly the radial excess and nothing else. Setting $\rho = 0$ gives $U(r) = \frac{1}{2} \eta_{\text{anc}} \|r\|^2$ — the quadratic residual regularizer of the backpropagated control — so the dead zone is a hinge generalization of that penalty, with ρ interpolating between them. Unlike a $\lambda \|r\|^2$ penalty — which competes with an unbounded gradient and loses (Section 3.3) — Equation (8) composes with the clipped reward field as a velocity, so the residual scale is set geometrically rather than by a loss-weight arms race. Heuristically, if the reward field were stationary, radially outward and saturated at $\|V_Q\| = \delta_Q$, balancing it against the anchor outside the dead zone gives $\delta_Q = \eta_{\text{anc}} (\|r\| - \rho)$, i.e. $\|r\| = \rho + \delta_Q / \eta_{\text{anc}}$. We use this only as a scale estimate when choosing η_{anc} : a learned critic’s field varies in direction and magnitude with r , so there may be several fixed points, cycles, or none, and we claim no unique equilibrium. pointwise pull $-\eta_{\text{anc}} r$, whose fixed point reproduces the DICE-RL anchor; the dead zone matters exactly when the reward field is weak (Section 5, LIBERO).

4.4 Relation to the backpropagated actor; critic

Proposition 1 (Small-step equivalence). At the current parameters, the gradient of Equation (4) is $\nabla_{\theta} \mathcal{L}_{\text{actor}} \propto -(\text{clip}_{\delta_Q}(V_Q) + V_{\text{anc}})^\top \partial r_{\theta} / \partial \theta$. With $V_Q = V_Q^{\text{grad}}$, no clipping ($\delta_Q, \delta_{\text{tot}} \rightarrow \infty$) and $\rho = 0$, this is the DICE-RL residual actor gradient with anchor weight η_{anc} .

CAST is therefore a strict generalization of its control: the new degrees of freedom are the pointwise action-space clip, the dead-zone anchor, and the resolution dial on V_Q — each of which we ablate.

Remark (not just clipped gradient ascent). One might object that Equation (4) with V_Q^{grad} is “DDPG with clipping.” The distinction is where the bound lives. Clipping ∇_{θ} bounds a parameter step, whose effect on the policy output is unknown and state-dependent; Equation (4) bounds the policy’s motion at every particle, which is the quantity that off-manifold failure depends on (Section 3.3). Moreover, only the regression form makes the two other components expressible at all: a dead-zone anchor and a value-only V_Q have no loss-gradient counterpart. The ablations in Section 5.5 test each distinction in isolation. The critic is unchanged from DICE-RL: ensemble TD with a CQL-style penalty pushing Q down at policy actions and up at data actions; field construction never leaks gradients into the critic (gradients w.r.t. actions only). Algorithm 1 summarizes one joint update.

5 Experiments

We test four claims. The first asks how the method compares to what a practitioner could use today. The second is the causal claim, and is the paper’s main result. The third and fourth ask which part of the method is responsible for it.

Algorithm 1 CAST: one update (V_Q^{grad} variant)

Require: frozen base π_0 ; residual r_θ ; critic Q_ϕ ; steps $\eta_Q, \eta_{\text{anc}}$; clips $\delta_Q, \delta_{\text{tot}}$; dead zone ρ ; particles K

- 1: sample batch of states s ; draw $z_i \sim \mathcal{N}(0, I)$, $a_i \leftarrow \pi_0(s, z_i) + r_\theta(s, z_i)$ $\triangleright$ no grad
- 2: $V_i \leftarrow \text{clip}_{\delta_Q}(\eta_Q \nabla_a \bar{Q}_\phi(s, z_i, a_i) / q_{\text{scale}})$ $\triangleright$ or top- k / tilted kernel field
- 3: $V_i \leftarrow V_i - \eta_{\text{anc}}(1 - \rho / \|r_i\|)_+ r_i$ $\triangleright$ dead-zone anchor, Equation (8)
- 4: $\tilde{r}_i \leftarrow \text{sg}[r_i + \text{clip}_{\delta_{\text{tot}}}(V_i)]$ $\triangleright$ policy moves $\leq \delta_{\text{tot}}$, always
- 5: $\theta \leftarrow \theta - \alpha \nabla_\theta \frac{1}{K} \sum_i \|r_\theta(s, z_i) - \tilde{r}_i\|^2$
- 6: update ϕ : ensemble TD + CQL penalty $\triangleright$ unchanged from Sun & Song (2026)

H1 CAST matches the strongest existing fine-tuners. The methods that, like it, let the critic displace actions — but with no action-space constraint — do not improve their base at all.

H2 Change only the actor update, and CAST gets more out of the same critic than backpropagating $-Q$ does.

H3 Take the action-space constraint away, and the update destroys the policy.

H4 The constraint has to act on the action target. The same shape placed in the loss, or on the parameter gradient, does not work.

Two supporting analyses follow: how much of the critic the update should consume (Section 5.6), and whether the result depends on the generator class the update is attached to (Section 5.7).

Throughout, “—” means not measured. No cell is inferred from a related run.

5.1 Setup

Benchmarks. Two, chosen because their critics differ. robomimic can and square are state-based, single-task, and densely covered, so the critic generalizes well; we report 300-episode evaluations averaged over the last three checkpoints, on 3 seeds. The eight hardest tasks of LIBERO-90 are image-based and sparsely rewarded, so the critic is far weaker; we report powered evaluation — 100 rollouts $\times$ 3 evaluation seeds. Both run inside the official DICE-RL harness with only the actor update replaced, so critic, replay, demonstrations and evaluation are byte-identical to the baseline. [TODO: appendix table: env details, horizons, demo counts, budgets.]

How we report success. Fine-tuners in this literature start from different pretrained policies, so absolute success confounds the fine-tuner with its base. Where methods do not share a base we give each method’s own base next to its result, and compare $\Delta = \text{fine-tuned} - \text{its own base}$. Only in Section 5.3, where every row is literally the same checkpoint, do we compare absolute numbers.

Methods. Every method on the drifting base shares the residual parameterization, critic, data and budget, so only the actor update varies.

- CAST — Equation (4) with the gradient reward field V_Q^{grad} .
- Backprop actor — the DICE-RL residual actor (Sun & Song, 2026): backpropagated $-Q$ with a filtered behaviour-cloning penalty, run on our drifting base. It reads the identical critic through the identical residual head, so any gap is attributable to the update form. Note that it already carries two constraints of its own: the BC penalty in its loss, and gradient-norm clipping at 1.0 on the actor. Both matter in Section 5.5.
- FM + DICE-RL — the published pipeline, unmodified, on its own flow-matching base.

- Classic diffusion-RL fine-tuners (DIPO, QSM, DQL, IDQL, AWR) — official implementations, unmodified, from their own diffusion base at their own published budget.
- CAST (top- k) and (tilted) — the same update with the reward field built from the critic’s rankings or its soft values instead of its gradient (Section 5.6).

Hyperparameters are shared across CAST and its variants, one recipe per benchmark and never per task: $\delta_{\text{tot}}=0.15$, $\eta_{\text{anc}}=1.0$, $\tau=1$, $k=4$, with reward step $\eta_Q=0.5$ on robomimic and $\eta_Q=1.0$ on LIBERO, and a dead-zone radius $\rho=0.05$ in dimension-independent rms units on LIBERO.

5.2 H1: where the method stands among existing fine-tuners

Claim. CAST matches the strongest existing fine-tuners, and the methods closest to it — those with no action-space constraint — do not improve their base at all.

Verdict. Both halves hold. We match the strongest pipeline rather than beating it, and we say so.

Against the published flow-matching pipeline we reach parity, not superiority: 0.912–0.930 against 0.923–0.934 across three seeds. What we can say is that we reach it from a base of 0.382.

The two finished rows above the line are the ones that matter for this paper. DIPO ends at 0.283 against its own base of 0.279 — four tenths of a point after 24M environment steps, inside evaluation noise. QSM collapses to exactly 0.000 in all six evaluation blocks.

We checked our own setup before reporting a collapse. Every hyperparameter matches the published configuration ($n_{\text{critic warmup}}=5$, actor/critic learning rates $10^{-5}/10^{-3}$, q_{grad} coefficient 10, $\tau=0.005$, buffer 5000, replay ratio 16, batch 1000), and it gets more training than published, not less (300 against 201 iterations). The failure is immediate rather than gradual: base level (0.284) at iteration 0, zero by iteration 10, unchanged for the next 290. Both losses converge (actor 0.002, critic 0.01) rather than diverging. That is the signature of a degenerate fixed point — once the policy stops succeeding every reward is zero, the critic correctly learns $Q \approx 0$ everywhere, and $\nabla_a Q$ carries no signal with which to recover. One caveat: our diffusion base scores 0.279 where the original authors report roughly 0.4, and a weaker base implies a worse early critic, so we do not claim this reproduces any published QSM number.

The five rows split cleanly, and the split is the point. Group them by what the critic is allowed to do. DIPO, QSM and DQL let the critic displace actions — through an action-space gradient step, a score matched to $\nabla_a Q$, or backpropagation through the policy — with nothing bounding the displacement. All three fail outright: two at exactly 0.000, one at +0.004 after 24M environment steps. AWR lets the critic only weight actions the policy already produced, and it is the one method that improves (0.407, +0.128): with no displacement there is nothing for a missing bound to break. IDQL sits between — it never displaces either, but its Q-based action selection still exploits critic error, and it ends below its base (0.041). The failure is therefore not “critic-guided fine-tuning is unstable,” and not our harness: it tracks precisely whether the critic controls where actions move. That is the quantity our constraint bounds. The displacing methods are the unconstrained corner of our own design space, measured externally in their authors’ code at $125\times$ our interaction — and Section 5.4 reproduces exactly their behaviour inside our system by switching our constraint off. What separates CAST from that family is not the signal, the budget, or the policy class; it is the bound. AWR’s positive result sharpens rather than weakens the claim: consuming the critic safely was never the hard part — weighting schemes have always done so — but they leave the critic’s most informative output, its action gradient, on the table (Section 5.6 measures what that costs on the harder task).

CAST as a drop-in on an existing fine-tuner. The rows above compare whole systems, which leaves open whether CAST’s advantage is the update rule or the rest of our pipeline. The cleanest test is to change nothing except the update. Taking DICE-RL on the same

pretrained diffusion base that every classic baseline uses, and switching only its actor from the residual update to CAST’s field-target update, raises 300-episode success from 0.861 to 0.937 — +7.6 points, from a one-line configuration change, on top of a method that was already working well. This is the sense in which CAST is a technique rather than a system: it is additive to an existing fine-tuner, not a replacement for one. The corresponding cell on a flow-matching base is not yet measured at matched base and budget, so we do not report it.

We tried to rescue the collapsing baselines, and could not. Reporting a method at settings that make it fail invites the objection that it is merely mis-tuned. So we re-tuned. For QSM we swept the score coefficient, for DQL the Q -loss weight η , for DIPO the action learning rate — in each case toward the value that would match our critic’s gradient scale. All three ended at 0.000: QSM and DIPO collapse from the base 0.279 within one evaluation and stay there, DQL flickers back once and then dies.

The reason is instructive, and it is the same point the paper makes. Two of those three knobs are loss-scale constants, and Adam is invariant to loss scale: changing QSM’s coefficient by $6.5\times$ moved the resulting policy displacement from 0.030 to 0.033, which is nothing. The knob that is not loss-scale — DIPO’s action learning rate, which acts directly on stored actions — did have an effect, and it destroyed a baseline that was working at 0.283. This is exactly the asymmetry we build on: constants in the loss do not control how far the policy moves in action space, and the one constant that does is a blunt instrument. A bound placed on the displacement itself is what these methods lack. We previously reported a “DIPO + displacement bound” row here; it enabled only the step clip (action_trust_radius) without the base-restoring anchor, which is the configuration Table 4 shows scores 0.000 — so it did not test what its name claimed and we have removed it. The arms that add CAST’s anchor to DIPO’s own code are running and will replace it.

5.3 H2: the update form is what converts critic signal into success

Claim. Change only the actor update, and CAST gets more out of the same critic than backpropagating $-Q$ does.

Verdict. Holds on both benchmarks, on every seed, on both bases. This is the paper’s main result.

Can is close to saturated and mainly checks that nothing breaks: CAST reaches 0.978–0.993 from a base of 0.877, the backprop actor 0.940–0.970. Square is the discriminative task. Its base solves only 0.382 of episodes; CAST adds 55 points to reach 0.91–0.93 on every seed, where the backprop actor reaches 0.86–0.91. Same critic, same data, same budget, different update: the field-target form extracts more, on every seed of both tasks.

Figure 4 shows the difference is in speed as much as in endpoint. CAST reaches 0.91 on every seed by 16–18K steps; the backprop actor gets there at 19–21K on two seeds and not at all within the window on the third, trails at the matched budget on every seed, and catches up only near 40K where both converge to 0.97–0.99. A bounded update buys a faster, never-worse path to the same asymptote.

Where the ablations live. CAST has four moving parts — a step clip δ_Q , a dead-zone anchor of radius ρ , the choice of how the critic scores candidates, and the target-regression form itself — and each is taken apart separately later in this section, so we list them once here rather than interleaving them with the headline result. Table 4 switches the clip and the anchor on and off independently (the full 2×2); Table 5 sweeps the dead-zone radius ρ from 0 to ∞ ; Table 6 moves the same constraint between the loss, the parameters and the action target, including a hard projection; Table 6 replaces the critic’s gradient with coarser signals (top- k , tilted); and Table 8 is a leave-one-out over the whole LIBERO recipe. Table 7 is not an ablation but a scope test on a different benchmark family.

LIBERO hard-8. Here we deploy CAST as a routing system: one frozen multitask drifting base, one small per-task residual head and critic fine-tuned on each task’s own interactions,

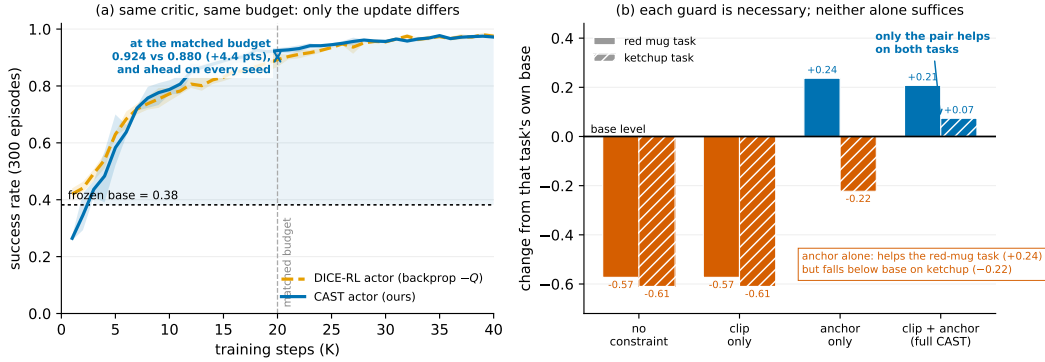


Figure 4: Why CAST wins, and why it needs both guards. (a) robomimic square, 300-episode evaluations; line is the seed mean, band the min-max over three seeds. Both arms share the base, critic, data, replay and budget — only the actor update differs. At the matched 20K budget CAST reaches 0.924 against the backpropagated actor’s 0.880, and is ahead on every seed; both converge near 40K, so the gain is in what the update extracts per step, not in the asymptote. (b) the constraint factorial, plotted as change from each task’s own base so that zero is “did nothing”. Removing the anchor is fatal whether or not the step clip is present (−0.57 and −0.61: both arms collapse to 0.000 success), which is the paper’s central negative result — many individually capped steps still compound into a walk off the data. The anchor alone survives but is not reliable: +0.24 on the red-mug task, −0.22 on ketchup, i.e. worse than not fine-tuning at all. Only the pair is positive on both. Neither guard is sufficient; the claim is about the pair.

and task-ID routing at deployment. This is the architecture every published LIBERO RL fine-tuner effectively uses. One recipe is shared across all eight tasks with no per-task tuning.

The verdict spans all eight tasks. On the drifting base CAST gains +3.9 to +23.9 points per task (mean +9.4 at $\eta_Q=2$, +7.7 at $\eta_Q=1$) and improves on the base for every one. The backprop actor, reading the same per-task critics, averages +0.9 and drops below base on the two highest-headroom tasks. The robomimic ordering transfers intact to the harder benchmark.

The same recipe on a flow-matching base is a wash. With all eight cells now measured, CAST on the flow-matching base averages 0.756 against that base’s 0.753: +0.3 points, three tasks up and five down. The two tasks it does improve are the two with real headroom — the ketchup task (+4.8) and the red-mug task (+9.1), both of which the drifting base also found hard — while on tasks the base already solves well it costs a little, most visibly −6.3 on the Study-3 caddy task. The flow-matching pipeline’s competing fine-tuners behave the same way: DICE-RL and GRPO also leave that base essentially where they found it.

We report this rather than restricting the claim to the base that flatters it. Two readings are consistent with the evidence and we cannot yet separate them. The first is headroom: the flow-matching base starts 9 points higher (0.753 vs 0.661) and is closer to what this critic and this budget can support, so there is less to win and more to break. The second is that the drifting base’s one-step noise-to-action map is unusually well matched to an update that displaces sampled actions, which would make part of our robomimic and LIBERO margin a property of the base rather than of the update alone. The drop-in result on a diffusion base (Table 1: 0.861 → 0.937) argues against the strong form of the second reading, since that base is neither drifting nor one-step. What we can state without qualification is the conditional: where the critic carries signal and the base leaves room, the update form is what converts one into the other.

Two implementation details were load-bearing and are ablated in Section 5.9. The dead zone must be measured in dimension-independent rms units: in vector-norm units the base

residual already exceeds ρ at LIBERO’s $S=112$ chunk dimension, so the anchor is active from the first update and freezes the residual. And the weaker critic gradient here rewards a doubled reward step, which the constraint makes safe to take: $\eta_Q=2$ lifts the mean from 0.738 to 0.755 and helps on six of eight tasks, with no setting on any task falling below its base.

Comparison to a critic-free policy gradient. On the one flow-matching task with real headroom, GRPO and CAST extract the same amount (0.883 and 0.878 on the red-mug task, both about +9.5) while the backprop actor manages +2.2. On the ketchup task CAST leads (0.738 against 0.700). On the tasks where nothing gains, all three sit together. So the separation on this benchmark is not CAST against the world; it is update form against backpropagation-through-the-critic. CAST matches what a policy-gradient method extracts without needing the noise-injection surrogate that makes likelihood ratios definable for a one-step generator in the first place. [TODO: per-task figure; remaining FM cells and matched-budget FM + GRPO.]

Where H2 stops: the shared-critic control. The alternative to routing is one policy and one critic trained on all eight tasks jointly. This flips the critic’s situation — our diagnostics show the shared critic’s action-gradient collapsing on most tasks, with per-task $\|\nabla_a Q\|$ spanning 0.003 to 0.55.

Nothing wins here, and that includes the competing method run on our own base. On the drifting base the backprop actor (0.655–0.668), CAST (0.655–0.659), both scoring variants (0.67–0.68) and DICE-RL at its own published settings (0.670–0.705) all sit within noise of the untuned base (0.671 ± 0.014). The point of the table is this column of near-zeros: it marks where our claim stops. DICE-RL’s +0.017 and CAST’s best variant at +0.006 are not distinguishable from each other at this seed spread: in the multitask setting we claim no advantage over it, and none is visible. The flow-matching pipeline looks stronger in absolute terms (0.735–0.756), but its own base already scores 0.738 ± 0.022 , so it too extracts essentially nothing; its apparent lead is inherited from pretraining. Applying our final recipe — every fix and hyperparameter behind the +9.4 — changes nothing here (0.668 against base 0.671).

This bounds H2 rather than contradicting it. The update form converts critic signal into success; where there is no usable signal there is nothing to convert. It is also worth noting that every drift-base arm preserves the base within noise, including the backprop actor — consistent with a critic that is action-blind rather than action-exploitable. There is no steep hallucinated gradient here to ride. Finally, a methodological note: single-seed 20-episode endpoints ranged 0.63–0.78 across these same arms, and powered evaluation collapses that spread to the base value. [TODO: per-task breakdown figure once the remaining seed evaluations land.]

5.4 H3: the action-space constraint is what makes the form viable

Claim. Take the action-space constraint away, and the update destroys the policy.

Verdict. The factorial separates the two guards cleanly, but Table 6 then shows the shape of the constraint matters more than the machinery that enforces it — including a loss-space hinge that beats us. Removing the base-restoring term costs everything — with the step clip alone, success is 0.000 on both tasks, identical to using no constraint at all. Removing the step clip instead is survivable but erratic (0.810 on the red-mug task, but 0.387 on the ketchup task, below that task’s base). Only the pair is reliable on both.

CAST bounds the target’s motion two ways: a pointwise clip on each particle’s displacement, and the dead-zone anchor. We claim neither over the other. The hypothesis is that some action-space constraint is necessary, so the experiment is the full factorial.

With no constraint at all, success is exactly 0.000 on both tasks, from bases of 0.573 and 0.610. The unconstrained action-gradient update destroys the policy. This reproduces inside our own system — our critic, our data, our budget — the behaviour DIPO and QSM show

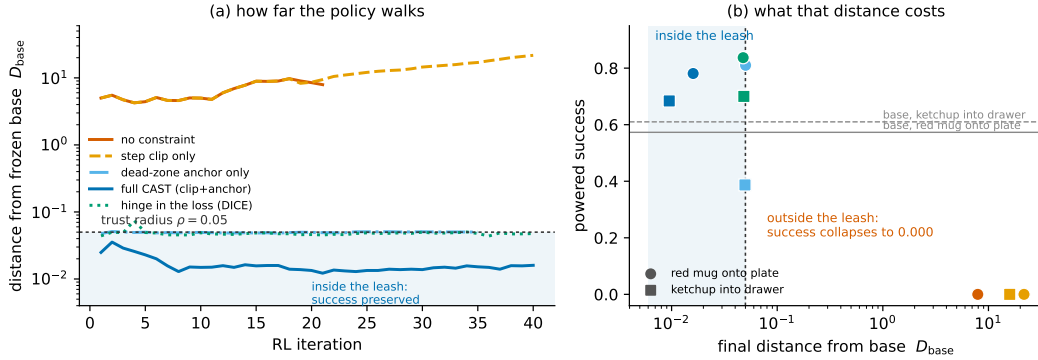


Figure 5: Distance from the base predicts collapse. (a) How far the policy walks from its frozen base during fine-tuning, $D_{\text{base}} = \mathbb{E} \|r_\theta\|_{\text{rms}}$, read from the training logs of the Table 4 arms on the red-mug task (log scale; curves end where the preemptible slot did). With no base-restoring term the policy leaves its base without bound, and the pointwise step clip does not prevent this — many individually-capped steps compound into a walk two orders of magnitude past the trust radius. (b) Why that matters: final distance against powered success, for both tasks. Every arm that ends inside the leash keeps or improves its base; every arm that ends outside scores exactly 0.000. This is the paper’s mechanism in one plot — the constraint is not a regulariser that trades a little performance for stability, it is the difference between a working policy and a dead one.

in their authors’ code at $125\times$ the interaction. That is the paper’s central negative result, and it is why we claim the constraint rather than the transport.

Figure 5 is where the mechanism becomes visible instead of inferred. Endpoint success tells us that constrained arms work; the residual trajectory tells us whether the policy was held near its base throughout, or merely happened to end near it.

The constraint’s shape is not critical; its presence is. Table 5 sweeps the dead-zone radius over two orders of magnitude. Every interior setting lands between 0.771 and 0.863 on the red-mug task and between 0.671 and 0.734 on the ketchup task — a spread no larger than this benchmark’s seed-to-seed variation.

The two endpoints behave differently, and the difference is the point. At $\rho=0$ the anchor becomes an always-on behaviour-cloning penalty and performance falls to roughly base level on both tasks: an always-on pull erases the residual. At $\rho\rightarrow\infty$ the dead-zone term goes inert, but this is not an unconstrained run — the weak pointwise restoring component and the displacement clip both remain, so it is a re-shaped constraint rather than an absent one. The genuinely unconstrained configuration is the first row of Table 4.

So a constraint must exist, and it must not be always-on. Between those two failure modes the setting is not critical, and we claim no preferred radius: $\rho=0.05$ sits inside a wide flat region, not at an optimum. The $\rho\rightarrow\infty$ cell is worth reading alongside Table 4, because together they say the same thing twice: that configuration still scores 0.803 because a weak pointwise restoring term survives in it, and when every restoring term is removed and only the displacement clip is left, success is 0.000. What the sweep varies is the shape of the tether; what Table 4 removes is the tether itself.

5.5 H4: the constraint has to act on the action target

Claim. The constraint has to act on the action target. The same shape placed in the loss, or on the parameter gradient, does not work.

Verdict. Two rows support it. The decisive row — giving the backpropagated actor our exact dead-zone shape inside its loss — is not yet run. We name it below as the experiment that would overturn this claim.

The most economical objection to this paper is that CAST is DICE-RL with a differently shaped regularizer, and that putting the constraint on the action target rather than in the loss is presentation rather than mechanism. Table 6 is built to test exactly that.

The table now separates where the constraint acts from what shape it takes, and only the first turns out to matter. Both action-target rows work and both loss-space rows fail: a hard projection onto $\|r\| \leq \rho$ reaches 0.790 and 0.733, our dead-zone anchor 0.781 and 0.684, while the always-on quadratic pull collapses to 0.549 and 0.601. We report the projection result even though it is the better of the two — on the ketchup task by 4.9 points, beyond that task’s seed spread. A simple hard cap on the residual is at least as good as the restoring field we build, which means our evidence supports the general claim (bound the displacement in action space) more strongly than it supports our particular instrument for doing so. That cap is also what ResFiT’s residual scale already implements, so the action-space constraint is not itself novel; what remains ours is the target-displacement form of the update, isolated in Table 2.

The first row is DICE-RL exactly as published, and it is worth being precise about what it already contains: a behaviour-cloning penalty in the loss and gradient-norm clipping at 1.0 on every actor step of every run in this paper. It constrains in both of the two places we say are wrong, and it still reaches only 0.550 and 0.587 against bases of 0.573 and 0.610.

The $\rho=0$ row makes the same point from inside our own method, and the correspondence is exact rather than analogous: by Proposition 1, CAST at $\rho=0$ with the clips removed is the DICE-RL residual actor’s update. Give the constraint the always-on quadratic shape that prior work uses and the policy returns to base level. This is the quantitative form of the toy’s finding that a penalty inside a loss must out-shout an unbounded gradient, and loses.

The sharpest objection is now measured, and it goes against the update form. DICE + hinge gives the backpropagated actor our dead-zone geometry inside its loss: same base, same critic, same $\beta_{BC}=100$, only actor_mode differs. We pre-registered the reading — if it matches CAST, the geometry is the whole story and the form is presentation. It does not merely match. It reaches 0.837 and 0.700 against full CAST’s 0.781 and 0.684 (and 0.627 on the matched arm we could verify), with its three evaluation seeds at 0.84/0.81/0.86 on the red-mug task. By our own criterion, on this benchmark the dead-zone geometry is doing the work, and the target-displacement form is not what earns the gain.

This does not erase Table 2 and Table 3, where the backpropagated actor without that geometry manages +0.9 against CAST’s +9.4. Read together, the three tables say something narrower than we set out to claim: a dead zone — a bounded region around the frozen policy with a free interior — is the ingredient that matters, and it can be delivered through a loss hinge, a hard projection, or our restoring field roughly equally well. Every configuration carrying that geometry lands in 0.68–0.84; both configurations without it (the L_2 penalty with gradient clipping, and the always-on pull at $\rho=0$) sit at base level or below, 0.549–0.601. What the target-displacement form buys is not a better ceiling but a way to get the geometry without differentiating through the critic at all — which is what makes it portable to the drop-in setting of Table 1, where the host method’s loss is not ours to edit.

5.6 How much of the critic should the update consume?

Claim. The critic’s gradient direction matters. CAST is not winning just by moving less.

Verdict. Holds on both benchmarks and both tasks.

Because reward enters only as a field, the critic can be consumed at lower resolution: transport toward its top- k ranked particles, or toward the cloud’s own $\exp(Q/\tau)$ -tilted density. Both delete the gradient direction and change nothing else.

The answer is the same wherever we ask it. On robomimic, where the critic generalizes, the value-only fields preserve can (0.91–0.95) but collapse on square, despite verified-healthy residuals and longer training. On LIBERO the same two substitutions fall to 0.577 and 0.697 on the red-mug task, and to 0.341 and 0.378 on the ketchup task — not merely below the full recipe but far below the untuned base, deficits of 34 and 31 points.

Two benchmarks, two bases, one conclusion: the ranking information a critic can supply is not a substitute for its gradient. CAST keeps the critic’s most informative signal and bounds only how far that signal may carry the policy in one step. Conservatism alone would not produce these numbers — it would produce the base.

5.7 Generality across generator classes

Claim. The update needs only a frozen noise-to-action map, so it transfers to other generator types.

Verdict. Holds on the four flow-matching tasks measured so far. Four remain, so we withhold a row mean.

On its own, stronger flow-matching base the published pipeline extracts essentially nothing per task: FM + DICE-RL scores -0.8 , -0.2 , $+2.2$ and -2.8 points relative to that base on the ketchup task, the orange-juice task, the red-mug task and the right-caddy task (mean -0.4) — the same near-zero extraction its actor shows on the drifting base ($+0.9$). Swapping only the actor update helps on three of the four: CAST on the identical base and budget beats FM + DICE-RL by $+5.6$ (ketchup task), $+6.9$ (red-mug task) and $+1.6$ (right-caddy task), reaching 0.738 and 0.878 on the first two, which is $+4.8$ and $+9.1$ over that base.

The exception is instructive. On the orange-juice task the flow-matching base already scores 0.917 and neither update improves it (-1.8 for CAST, -0.2 for DICE-RL). More generally, the never-below-base behaviour CAST shows on all eight drifting-base tasks does not carry over here: on the orange-juice task and the right-caddy task it ends 2.0 and 1.2 points below where it started. Fine-tuning a base that is already near its ceiling is a different problem from lifting a weak one, and our trust region is calibrated for the latter.

5.8 Scope: when is the constraint needed, and does it transfer?

The mechanism sections establish that the constraint is necessary in our setting. Three scoping questions then determine how far the claim reaches, and we test each with update rules already validated in this paper — the field-target update and the backpropagated DICE-RL actor — rather than with methods that fail everywhere and so can distinguish nothing.

Q1: is the failure specific to fine-tuning from narrow data? The collapse mechanism needs a critic whose competence is a thin tube around demonstrated behaviour. We therefore cross the same update rules over data regimes: narrow-coverage fine-tuning (LIBERO, robomimic; measured throughout this paper) against a broad-coverage base — a DMC cartpole base distilled deliberately from multiple noisy SAC checkpoints, so the critic’s training tube is wide. The residual question inside Q1 is already settled by Table 1: DIPO, QSM and DQL fine-tune the full policy with no residual head and collapse regardless, so the residual parameterization is packaging, not disease.

Q2: does the failure need sparse reward? The same DMC base supports a controlled pair — swingup (dense) and swingup_sparse — identical dynamics, identical base, identical critic architecture, only the reward signal differing. Table 7 runs three arms on each. The registered prediction, from the degenerate-fixed-point analysis of Section 5.2, was that dense reward lets a drifted policy recover while sparse reward does not, and the completed runs bear it out in the direction that matters: under dense reward the unconstrained update not only survives but ends above its base in interim evaluations (~ 540 against a base of ~ 430), the exact opposite of its sparse-manipulation behaviour. The correct reading of sparsity is therefore not that it causes the drift — the drift is the update’s doing — but that it

removes the road back: once off-manifold with no successes, every reward is zero and the critic has nothing with which to guide a return. Dense reward keeps that road open. This also resolves an apparent paradox in our own results: our sparse-reward benchmarks are survivable for constrained methods precisely because the constraint prevents the one-way step through the trapdoor that sparsity cannot undo.

How CAST is configured here. We do not re-tune for DMC. The arm runs the robomimic recipe unchanged: actor_mode field-pointwise, V_Q from the critic gradient with reward step $\eta_Q=0.5$, anchor step 0.05, total displacement clip $\delta_{\text{tot}}=0.15$, restoring step 1.0, $K=16$ noise samples per state, 3-step returns, and a one-step (flow_steps=1) drifting base distilled from the SAC checkpoints. The dead-zone radius is $\rho=0$ here, i.e. the restoring term is always on rather than gated — the setting Table 5 shows is worst on LIBERO. The DMC numbers are therefore an untuned transfer, not a best case, and should be read as a scope test rather than a performance claim.

Q3: does the bound transfer to other algorithms? Two transfers are tested, both on validated hosts. First, the dead-zone geometry inside the backpropagated actor’s loss — the DICE + hinge row of Table 6 — asks whether our constraint helps the strongest existing update when expressed in its native form. Second, as an external probe, DIPO’s own action-improvement loop is given a per-round cap on total displacement and a consistent buffer (two config-gated switches in its official code, byte-identical when off); DIPO is the weakest member of the family (+0.004 in Table 1), so any recovery is attributable to the bound alone. Both runs are in progress and will be reported whichever way they land.

5.9 Robustness, and what the bound actually does

The remaining knobs are task-dependent, and they track the critic rather than the task’s difficulty. On the ketchup task, whose critic carries a strong action-gradient ($\|\nabla_a Q\| \approx 0.55$), 3-step returns are worth 17 points and a doubled reward step 4.5, while widening the movement bound alone does nothing: this task wants more reward signal, not more room to move. On the red-mug task, whose gradient is an order of magnitude weaker (≈ 0.035), the pattern inverts — returns and step size are within noise, but widening the trust region is worth +9.3 points. We report this as a regime difference, not a tuned optimum: it is one seed, and the red-mug task carries a ± 9 -point spread, so the direction is trustworthy and the magnitude is not.

Is the gap just a better step size? We tested this directly on the red-mug task by sweeping each method’s own step-size knob under the identical protocol. The backpropagated actor’s learning rate over a $10\times$ range gives 0.550, 0.548 and 0.578 — every setting at or below the untuned base (0.573). Our reward step over an $8\times$ range gives 0.775, 0.817, 0.908 and 0.750 for $\eta_Q \in \{1, 2, 4, 8\}$ — every setting 18–33 points above that same base. The two bands do not overlap at any step size either method was given. A larger step does not turn the backpropagated update into the field update. (Both sweeps are single-seed, so the ordering within each band should not be read as a tuned optimum; the band separation, at 18 points minimum, is well outside seed noise.)

How often does the bound actually bind? The clip only ever reduces a displacement, so whenever a particle’s proposed motion is already shorter than δ_{tot} the update is exactly the unbounded one. In the Section 3.3 setting at our operating point, the clip is active on 17.3% of sampled actions and the median proposed displacement is only $0.12\delta_{\text{tot}}$. Tightening the cap to 0.05 raises the binding rate to 26.3%; loosening it to 1.0 drops it to 4.8%.

These numbers say what kind of object the trust region is. It is not a step-size controller: for most proposals the critic’s own magnitude still sets the distance, exactly as in the unconstrained update. What the constraint governs is the tail — the minority of proposals where an imperfect critic asks for a displacement far outside anything its data supports, and which, left alone, are enough to end the run (Table 4, first row). That is why our claim is about where the critic’s proposal is mediated, not about how far the policy travels

on average. It is also why we do not adjudicate between the two implementations of the constraint: both truncate the same tail.

[TODO: Remaining: robomimic dispersion retention (action dispersion and best-of- N before and after fine-tuning against the backprop control — does the field update preserve the multimodality the base was trained for?).]

6 Conclusion

This paper is about one narrow, common regime: post-training a generative policy that was pretrained on a small set of demonstrations, online, under sparse binary reward. Our experiments locate the boundary of that regime from both sides. Inside it, any update that lets the critic displace actions without an action-space bound destroys the policy — in our system and in the official implementations of an entire method family — because the critic is only competent in the narrow tube its data covers and sparse reward offers no route back once the policy leaves it. Outside it — dense reward, or broad-coverage training — the same unconstrained updates survive and can even gain, so we claim nothing there. Within the regime, the missing update rule is field-target regression under an action-space bound anchored to the frozen base. CAST expresses reward and anchoring as composable velocity fields over action particles, and recovers the backpropagated direction in the small-step limit. Its working component is the action-space trust region: the critic sets the direction, while a bound — a pointwise clip on each particle’s displacement, a restoring pull outside a radius of the frozen base, or both — sets how far any single update may carry the policy.

The experiments locate the advantage precisely. Given the same base, critic, data and budget, the field-target form extracts far more than backpropagating that critic: +55 points on robomimic square, and +9.4 versus +0.9 on average across the eight hardest LIBERO-90 tasks, where the backpropagated actor falls below its base on the two highest-headroom tasks and CAST improves on every one. The same ordering holds when the base is swapped for the stronger flow-matching policy, so the effect is attributable to the update rule rather than to the starting point. Against a critic-free policy-gradient baseline at matched interaction budget, CAST is at parity (+10.1 versus +9.5 per-task mean on LIBERO; 0.878 versus 0.883 on the flow-matching base’s one high-headroom task) — while requiring none of the noise injection that makes likelihood ratios definable for a deterministic one-step generator in the first place. Ablations confirm the mechanism rather than the hyperparameters, and they constrain what we may claim. Replacing the dead-zone with an always-on penalty ($\rho=0$, exactly a behaviour-cloning term) falls back to base level (0.549 and 0.601 against bases of 0.573 and 0.610), so the constraint must be shaped in action space rather than added to the loss. Switching the two guards on and off independently separates them cleanly. Removing both collapses success to exactly 0.000 on both tasks; keeping only the pointwise step clip gives exactly 0.000 as well, no better than no constraint at all. The base-restoring term is what makes the update viable (0.810 on the red-mug task with the clip removed), though on its own it is unreliable across tasks (0.387 on the ketchup task), and the two together are what hold on both. The constraint that matters is therefore not a limit on how fast the policy moves but a tether to where it started.

Limitations. Three specific results run against our thesis and we state them plainly. On the Study-3 caddy task the backpropagated actor ends above CAST (0.653 versus 0.635); on two flow-matching tasks the un-finetuned base beats every fine-tuner including ours (0.917 on the orange-juice task, 0.620 on the right-caddy task); and on robomimic can the flow-matching pipeline edges us on two of three seeds. Single-seed variation is also substantial on LIBERO — repeating the frozen recipe on the red-mug task with a second seed moves the result by 8.9 points — so per-task deltas smaller than roughly ten points, including our η_Q and δ_{tot} sweeps, indicate direction rather than magnitude. The claims that survive this variation are the ones separated by margins far larger than it: the +9.4 versus +0.9 mean gap, the non-overlapping step-size bands, and the below-base collapse when the critic’s gradient is deleted.

The gains track how much a critic actually knows. When a single critic is shared across all eight LIBERO tasks its action-gradient collapses and no critic-based update — ours

or the state of the art’s — improves on its base; when a base is already near its ceiling, CAST can end slightly below it (-2.0 and -1.2 on two flow-matching tasks), since our trust region is calibrated for lifting weak bases rather than for refining strong ones. Our critic-trust diagnosis is also post hoc: choosing the reward-field resolution online from critic-trust signals is the natural next step, as is testing whether the reward step should adapt per task. Finally, all results are state-based simulation on a single embodiment.

We should be precise about which part of the story generalizes, because two claims are easy to conflate. The update rule is base-agnostic: it consumes a frozen noise-to-action map, a residual head and a critic, and never touches how the base draws its action, and it transfers unchanged to a diffusion base, where dropping it into DICE-RL raises success from 0.861 to 0.937 . Its size of benefit, however, is not base-agnostic: on the flow-matching LIBERO base the same recipe is a wash against the base (0.756 vs 0.753) and only $+1.3$ points over DICE-RL across the seven tasks where both ran. What does not generalize is the economy of the construction. A drifting model is pretrained by field-target regression, so on that base the fine-tuner reuses machinery the policy class already has; a flow-matching model is pretrained by conditional velocity regression, so there the same update is bolted on rather than inherited (and costs one base forward pass per particle per integration step). The principle we can defend is therefore narrow: what keeps a critic-driven update on the data is a bound on the total displacement in action space, not a limit on step size. We are deliberately not claiming more than the location of the constraint. A hard projection onto $\|r\| \leq \rho$ does as well as our dead-zone anchor and better on one task (Table 6), so the anchor is one workable instrument rather than the necessary one. The distinction is not cosmetic. A step limit constrains speed, and a drift assembled from permitted small steps still arrives; in our isolation the limit alone leaves the policy at zero success. A behaviour-cloning penalty restores, but does so everywhere, including the region where the critic is reliable, and must therefore trade far-field containment against near-field progress with a single coefficient; it also fails alone. The dead zone separates those jobs.

We deliberately claim no more than this. We have no principled rule for choosing ρ — it is tuned, not derived — and the argument for why the dead zone should exist is an argument about matching the regularizer’s spatial profile to the critic’s reliability, not a bound. Deriving ρ from a measure of local critic reliability is the obvious next question, and we leave it open.

Reproducibility statement

[TODO: Code release (training framework + official-harness implant), exact configs (family-shared recipe), seeds, eval protocols (300-ep robomimic; 100×3 powered LIBERO), compute (single L40S/A100 per run, 8–20h), toy experiment script. All numbers in Table 2 are averages, never best-of.]

References

- Abbas Abdolmaleki, Jost Tobias Springenberg, Yuval Tassa, Remi Munos, Nicolas Heess, and Martin Riedmiller. Maximum a posteriori policy optimisation. In International Conference on Learning Representations (ICLR), 2018.
- Kevin Black, Michael Janner, Yilun Du, Ilya Kostrikov, and Sergey Levine. Training diffusion models with reinforcement learning. In International Conference on Learning Representations (ICLR), 2024. arXiv:2305.13301.
- Wonhyeok Choi, Shutong Ding, Minwoo Choi, Jungwan Woo, Kyumin Hwang, Jaeyeul Kim, Ye Shi, and Sunghoon Im. A review of online diffusion policy rl algorithms for scalable robotic control, 2026. URL <https://arxiv.org/abs/2601.06133>.
- Mingyang Deng, He Li, Tianhong Li, Yilun Du, and Kaiming He. Generative modeling via drifting, 2026. URL <https://arxiv.org/abs/2602.04770>.
- Ying Fan, Olivia Watkins, Yuqing Du, Hao Liu, Moonkyung Ryu, Craig Boutilier, Pieter Abbeel, Mohammad Ghavamzadeh, Kangwook Lee, and Kimin Lee. DPOK: Reinforce-

- ment learning for fine-tuning text-to-image diffusion models. In Advances in Neural Information Processing Systems (NeurIPS), 2023.
- Scott Fujimoto, Herke van Hoof, and David Meger. Addressing function approximation error in actor-critic methods. In International Conference on Machine Learning (ICML), 2018.
- Scott Fujimoto, David Meger, and Doina Precup. Off-policy deep reinforcement learning without exploration. In International Conference on Machine Learning (ICML), 2019.
- Tuomas Haarnoja, Haoran Tang, Pieter Abbeel, and Sergey Levine. Reinforcement learning with deep energy-based policies. In International Conference on Machine Learning (ICML), 2017.
- Philippe Hansen-Estruch, Ilya Kostrikov, Michael Janner, Jakub Grudzien Kuba, and Sergey Levine. IDQL: Implicit Q-learning as an actor-critic method with diffusion policies. arXiv preprint arXiv:2304.10573, 2023.
- Ayush Jain, Norio Kosaka, Xinhui Li, Kyung-Min Kim, Erdem Biyik, and Joseph J. Lim. Mitigating suboptimality of deterministic policy gradients in complex q-functions, 2025. URL <https://arxiv.org/abs/2410.11833>.
- Dmitry Kalashnikov, Alex Irpan, Peter Pastor, Julian Ibarz, Alexander Herzog, Eric Jang, Deirdre Quillen, Ethan Holly, Mrinal Kalakrishnan, Vincent Vanhoucke, and Sergey Levine. QT-Opt: Scalable deep reinforcement learning for vision-based robotic manipulation. In Conference on Robot Learning (CoRL), 2018.
- Juil Koo, Mingue Park, Jiwon Choi, Yunhong Min, and Minhyuk Sung. Drifting field policy: A one-step generative policy via wasserstein gradient flow, 2026. URL <https://arxiv.org/abs/2605.07727>.
- Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. Conservative Q-learning for offline reinforcement learning. In Advances in Neural Information Processing Systems (NeurIPS), 2020.
- Haozhan Li, Yuxin Zuo, Jiale Yu, Yuhao Zhang, Zhaohui Yang, Kaiyan Zhang, Xuekai Zhu, Yuchen Zhang, Tianxing Chen, Ganqu Cui, Dehui Wang, Dingxiang Luo, Yuchen Fan, Youbang Sun, Jia Zeng, Jiangmiao Pang, Shanghang Zhang, Yu Wang, Yao Mu, Bowen Zhou, and Ning Ding. Simplevla-rl: Scaling vla training via reinforcement learning, 2025. URL <https://arxiv.org/abs/2509.09674>.
- Zechu Li, Rickmer Krohn, Tao Chen, Anurag Ajay, Pulkit Agrawal, and Georgia Chalvatzaki. Learning multimodal behaviors from scratch with diffusion policy gradient, 2024. URL <https://arxiv.org/abs/2406.00681>.
- Yang Liu, Prajit Ramachandran, Qiang Liu, and Jian Peng. Stein variational policy gradient. arXiv preprint arXiv:1704.02399, 2017.
- Guozheng Ma, Lu Li, Haoyu Wang, Zixuan Liu, Pierre-Luc Bacon, and Dacheng Tao. What makes value learning efficient in residual reinforcement learning?, 2026. URL <https://arxiv.org/abs/2602.10539>.
- Max Sobol Mark, Tian Gao, Georgia Gabriela Sampaio, Mohan Kumar Srirama, Archit Sharma, Chelsea Finn, and Aviral Kumar. Policy agnostic rl: Offline rl and online rl fine-tuning of any class and backbone, 2024. URL <https://arxiv.org/abs/2412.06685>.
- David McAllister, Songwei Ge, Brent Yi, Chung Min Kim, Ethan Weber, Hongsuk Choi, Haiwen Feng, and Angjoo Kanazawa. Flow matching policy gradients, 2025. URL <https://arxiv.org/abs/2507.21053>.
- Mitsuhiko Nakamoto, Yuexiang Zhai, Anikait Singh, Max Sobol Mark, Yi Ma, Chelsea Finn, Aviral Kumar, and Sergey Levine. Cal-QL: Calibrated offline RL pre-training for efficient online fine-tuning. In Advances in Neural Information Processing Systems (NeurIPS), 2023.

Seohong Park, Qiyang Li, and Sergey Levine. Flow Q-learning. arXiv preprint arXiv:2502.02538, 2025.

Xue Bin Peng, Aviral Kumar, Grace Zhang, and Sergey Levine. Advantage-weighted regression: Simple and scalable off-policy reinforcement learning. arXiv preprint arXiv:1910.00177, 2019.

Michael Psenka, Alejandro Escontrela, Pieter Abbeel, and Yi Ma. Learning a diffusion model policy from rewards via q-score matching, 2025. URL <https://arxiv.org/abs/2312.11752>.

Allen Z. Ren, Justin Lidard, Lars L. Ankile, Anthony Simeonov, Pulkit Agrawal, Anirudha Majumdar, Benjamin Burchfiel, Hongkai Dai, and Max Simchowitz. Diffusion policy optimization. In International Conference on Learning Representations (ICLR), 2025. arXiv:2409.00588.

Idan Shenfeld, Jyothish Pari, and Pulkit Agrawal. RL’s razor: Why online reinforcement learning forgets less, 2025. URL <https://arxiv.org/abs/2509.04259>.

Zhanyi Sun and Shuran Song. From prior to pro: Efficient skill mastery via distribution contractive rl finetuning, 2026. URL <https://arxiv.org/abs/2603.10263>.

Ziyu Wang, Alexander Novikov, Konrad Zolna, Josh Merel, Jost Tobias Springenberg, Scott Reed, Bobak Shahriari, Noah Siegel, Caglar Gulcehre, Nicolas Heess, and Nando de Freitas. Critic regularized regression. In Advances in Neural Information Processing Systems (NeurIPS), 2020.

Tonghe Zhang, Chao Yu, Sichang Su, and Yu Wang. Reinflow: Fine-tuning flow matching policy with online reinforcement learning, 2026. URL <https://arxiv.org/abs/2505.22094>.

Zhiyuan Zhou, Andy Peng, Charles Xu, Qiyang Li, Tobias Springenberg, Kevin Frans, and Sergey Levine. Test-time gradient guidance of flow policies in reinforcement learning, 2026. URL <https://arxiv.org/abs/2606.11087>.

A Additional results

[TODO: Full seed tables, training curves, hard-8 per-task breakdown.]

B Hyperparameters

[TODO: Family-shared recipe table; official DICE-RL hypers; sweep grids.]

C The kernel transport operator

[TODO: Full form of $\mathcal{V}$: global distance scale, per-radius softmax-softmax affinity, contested-attractor weighting, self-exclusion, per-radius normalization on/off (restoring vs. constant-pull), radii $\{0.02, 0.05, 0.2\}$. Port from `drift_field.py` docstrings.]

D Toy two-regime construction

[TODO: Details of the hard-critic/easy-critic toy; failure of compass updates.]

D.1 Reward-step sweep

CAST’s reward step η_Q is the one hyperparameter we varied. We report the whole sweep here because the per-task optima disagree, and because the main tables deliberately do not use them: every CAST number in Table 3 runs a single shared η_Q across all eight tasks, the same courtesy the baselines get by being reported at their published recipes. Tuning per

task would raise our numbers — 0.880 instead of 0.802 on the red-mug task — and would not be a like-for-like comparison.

The flow-matching rows also correct an earlier reading of ours. At $\eta_Q=1$ CAST sits below the base on the front-caddy task (0.667 against 0.683) and gains only +0.003 on the eight-task mean; at $\eta_Q=2$ the same task gains +0.100. The flow-matching column was therefore under-tuned rather than resistant to the method, and we had previously attributed its flat result to limited headroom — an explanation the per-task numbers do not support, since gains are the same (+0.003 mean) for tasks whose base is below 0.72 and above it.

D.2 Measured diagnostics behind the mechanism claims

Three claims in the main text rest on direct measurements rather than on outcomes. We give the numbers here so they can be checked.

(i) Why nothing works in the multitask setting. Table 13 reports that no update rule moves the shared-critic base. The cause is upstream of every method: the shared critic barely changes its value when the action moves, so both mechanisms these methods rely on — a gradient to climb, and a spread over candidates to select from — are starved at once.

(ii) Why QSM collapses on the first update. QSM’s actor loss retrains the denoiser to emit $\eta \nabla_a Q$ in place of the noise it was pretrained to predict, with no behaviour-cloning term. Whether that is survivable depends on whether the two are on the same scale. They are not: measured on the pretrained square base with a freshly initialised critic, $\|\nabla_a Q\| = 0.0127$ against $\|\epsilon\| = 5.212$. At QSM’s published coefficient the target is 2.4% of what the denoiser currently outputs, so the pretrained map is overwritten rather than adjusted — consistent with the observed $0.284 \rightarrow 0.000$ within a single evaluation interval. No coefficient resolves this: reaching parity needs $\eta \approx 400$, at which point the denoiser is copying a random critic. (scripts/diag_qsm_scale.py reproduces this in seconds.)

(iii) How far each constraint actually lets the policy travel. Figure 5 plots this; the end-points are given here. The trust radius is $\rho = 0.05$. Note that the step clip alone permits the largest excursion of all — many individually capped steps compound — and that full CAST stops at roughly a third of its own allowance, which is what the loosening sweep in Table 9 probes.

D.3 The multitask setting

Table 3 fine-tunes each task with its own residual head and critic, routed by task ID at deployment — the architecture published LIBERO RL fine-tuners use. The alternative is one policy and one critic trained across all eight tasks jointly. We ran it, nothing works, and we report it here rather than in the main text because the result is a property of the setting rather than of any method: with a shared critic there is no usable action signal for an update rule to convert, so every arm lands within seed noise of its base. Table 10 is the measurement — 16 candidate actions scored within 0.7% of one another, and a 0.05 action step moving Q by under one part in forty.

Table 1: Fine-tuners on robomimic square, each from its own pretrained base. The classic diffusion-RL methods (DIPO, QSM, DQL, IDQL, AWR) are included as adapted for this exact setting — sparse binary-reward robomimic manipulation, fine-tuning a pretrained diffusion policy — by the DPPO authors, whose published baseline protocol and configurations we run verbatim; their original papers target dense-reward locomotion or offline D4RL, so the adapted versions are the ones that qualify for this benchmark. They run from official implementations without modification, at their published settings, which give them $300 \times 400 \times 50 \times 4 \approx 24\text{M}$ environment steps against our 192K — roughly $125\times$ our interaction. This is deliberately generous to them and is not a matched-budget comparison. They are also robomimic-only: their official implementations do not support LIBERO, so the LIBERO comparison in Table 3 is carried by FM + DICE-RL and GRPO instead. Success uses the same 300-episode protocol throughout. The lower block is the drop-in test, read in pairs: each base appears twice, once with DICE-RL’s own actor and once with that actor replaced by CAST, so the difference within a pair isolates the update rule. Concretely for the diffusion pair: DICE-RL and DICE-RL + CAST share the same pretrained diffusion base, the same critic, data, replay and budget, and differ in a single configuration line (actor_mode: residual vs field_pointwise). We also attempted to re-tune the three collapsing methods rather than report them at published settings; that attempt is reported in Section 5.2 and made them worse, so the published settings stand. What the shared base does and does not license. All five classic rows load a pretrained diffusion checkpoint from the DPPO release (square_pre_diffusion_mlp_ta4_td20/2024-07-10_01-46-16) that is absent from both projects’ public artifacts, so we substituted our own retrain at the architecture those configs specify (time_dim= 32), which reaches 0.279 against DPPO’s published square base of roughly 0.4–0.5. Because every one of these rows starts from that same checkpoint, the comparison among them — which is the only thing we argue from — is internally valid. What it does not license is any claim to reproduce a published number for these methods, and we make none: a weaker base implies a weaker early critic, which can produce a collapse that a stronger base would not. We are retraining that base across seeds to close the gap. [‡]DPPO now trains from the same checkpoint as every other diffusion row (square_diff_baselinearch_42); its earlier 0.563 $\rightarrow$ 0.876 used a different pretrained policy and is superseded. The retrain is still in progress — 0.561 at 368K of the $\sim 12\text{M}$ environment steps its schedule needs — so its fine-tuned cell is a lower bound and will rise. One measurement complicates the base column and we state it rather than hide it. The diffusion rows all load one file, but they do not all score it the same: the classic baselines’ harness reads it as 0.279 and DPPO’s reads the same file as 0.520. We traced this to the samplers rather than the weights — each method wraps the pretrained denoiser in its own sampling loop — but removing the obvious candidate (the 0.1 noise floor those loops impose) moved the number by 0.001, so the mechanism is not yet identified. Each row therefore reports its base as its own harness measures it, and Δ is always within-harness; cross-row base comparison is not meaningful. The flow-matching base measures 0.570 on square (300 episodes). “—” = not measured.

method	policy class	own base	fine-tuned	Δ
DIPO (Li et al., 2024)	diffusion	0.279	0.283	+0.004
QSM (Psenka et al., 2025)	diffusion	0.279	0.000	−0.279
DQL	diffusion	0.279	0.000	−0.279
IDQL	diffusion	0.279	0.041	−0.238
AWR	generic	0.279	0.407	+0.128
DPPO (Ren et al., 2025) [‡]	diffusion	0.520	0.561 [‡]	+0.041
adding CAST to DICE-RL, paired within each base				
diffusion + DICE-RL (Sun & Song, 2026) [§]	diffusion	0.279	0.861	+0.582
diffusion + DICE-RL + CAST [§]	diffusion	0.279	0.937	+0.658
drifting + DICE-RL	one-step drifting	0.382	0.861–0.909	+0.48–0.53
drifting + DICE-RL + CAST	one-step drifting	0.382	0.912–0.930	+0.53–0.55
flow matching + DICE-RL (Sun & Song, 2026)	flow matching	0.570	0.831 [¶]	+0.261
flow matching + DICE-RL + CAST	flow matching	0.570	0.934 [¶]	+0.364

Table 2: Matched actor-update comparison on robomimic (300-episode evaluations, last-3-checkpoint average). s42/s43/s44 are the three training seeds; each is an independent fine-tuning run from the same pretrained base, so the three numbers in a block show run-to-run spread, not three different tasks. Every row shares the same base checkpoint, critic, data, replay and budget — the only thing that changes between the last two rows is how the critic’s signal reaches the policy. Row 2 is DICE-RL’s own actor, which differentiates the critic through the policy — the standard choice and our baseline. Row 3 replaces only that actor with CAST, which displaces sampled actions along the critic’s gradient, bounds the displacement in action space, and regresses the policy onto the result. Every row here is a complete method, not an ablation. Rows 2 and 3 are the same system end to end — same frozen base, same critic, same replay, same budget, same number of gradient steps — and differ only in the boxed column. Nothing is removed from CAST in this table. [†] these two runs ended before 20K steps, so their last-three averages are taken at 17–19K (s42) and 14–16K (s43); both curves are flat there, so the earlier window if anything favours the baseline.

actor update	how the critic is used	can			square		
		s42	s43	s44	s42	s43	s44
frozen base, no RL	critic never queried		0.877			0.382	
DICE-RL actor (baseline)	Q differentiated through the policy	0.970 [†]	0.946 [†]	0.940	0.870	0.909	0.861
CAST actor (ours)	Q only queried; targets displaced, bounded, regressed	0.988	0.993	0.978	0.912	0.930	0.929

Table 3: LIBERO hard-8, per-task fine-tuning: one frozen multitask base, one small residual head and critic trained on each task’s own interactions, task-ID routing at deployment. Powered evaluation (100 rollouts $\times$ 3 evaluation seeds); CAST rows average the last three training checkpoints, and two training seeds on the ketchup, orange-juice and red-mug tasks. One recipe across all eight tasks, no per-task tuning: every CAST cell uses a single shared reward step, chosen for its mean over the eight tasks rather than per task. The full sweep is Table 9 — per-task optima disagree ($\eta_Q=4$ is best on the red-mug task, $\eta_Q=1$ on ketchup), and reporting each task at its own optimum would raise our numbers while giving us a tuning budget the baselines do not get. Task names are the LIBERO-90 scene and instruction; indices in that suite are 8, 21, 32, 53, 65, 73, 75, 81 in table order. Within each block every column shares that block’s base, so “CAST” in the left block means CAST fine-tuning the drifting base and in the right block the flow-matching base; across blocks only same-block comparisons are meaningful. “backprop” is the backpropagated actor (DICE-RL’s update on this base). $CAST_{\eta_1}$ and $CAST_{\eta_2}$ are CAST at reward step $\eta_Q=1$ and $\eta_Q=2$ — the single hyperparameter we varied, and the only place in this table where a method appears at two settings. The like-for-like cross-base comparison is therefore $\eta_Q=1$ against $\eta_Q=1$: that setting is shared by the drifting and flow-matching columns, and gives means of 0.738 and 0.756. We report $\eta_Q=2$ on the drifting base because we ran it there and it is better (0.755), but the matching flow-matching run does not yet exist, so the $\eta_Q=2$ column should not be read across bases. [‡]these four GRPO cells are provisional and not comparable to the other four. They come from a shorter run (40 vs 200 iterations) scored by a single 20-rollout deterministic evaluation, not the 100-rollout $\times$ 3-seed powered protocol used everywhere else in this table; the matched-recipe runs are queued and will replace them. We show them marked rather than blank, The GRPO row mean of 0.717 therefore mixes protocols and is marked; over the four matched-protocol cells alone it is 0.771. Within each base block, bold marks the best fine-tuning method on that row; the no-RL base is a reference rather than a competitor and is never bolded. The two blocks use different bases, so bolding is never applied across them — and note that on three flow-matching rows every method is below its own base, which the bold does not indicate. Across the five tasks run with two seeds the seed spread is 0.000–0.089, so only gaps above ~ 9 points are meaningful on the red-mug task and above ~ 2 points elsewhere. “—” = unrun cell: CAST’s eight cells are complete on both bases, while the DICE-RL and GRPO rows on the flow-matching base still have gaps, so their row means are withheld. The multitask (single-policy) version of this comparison is Table 13.

task	drifting base				flow-matching base				
	base	backprop	$CAST_{\eta_1}$	$CAST_{\eta_2}$	base	DICE	GRPO	$CAST_{\eta_1}$	$CAST_{\eta_2}$
Kitchen-1: open drawer, put bowl in	0.818	0.820	0.831	0.857	0.897	0.873	0.800 [‡]	0.900	0.887
Kitchen-3: turn on stove, put pan on	0.690	0.703	0.737	0.752	0.700	0.730	0.700 [‡]	0.690	0.727
Kitchen-5: ketchup into top drawer	0.610	0.587	0.694	0.729	0.690	0.682	0.700	0.738	0.773
Living-2: orange juice into basket	0.785	0.800	0.842	0.853	0.917	0.915	0.897	0.897	0.913
Living-5: red mug onto plate	0.573	0.550	0.826	0.812	0.787	0.809	0.883	0.878	0.887
Study-1: book into front caddy slot	0.648	0.663	0.710	0.752	0.683	0.690	0.600 [‡]	0.667	0.783
Study-1: book into right caddy slot	0.578	0.583	0.657	0.648	0.620	0.592	0.603	0.608	0.563
Study-3: book into front caddy slot	0.588	0.653	0.607	0.635	0.733	0.708	0.550 [‡]	0.670	0.680
mean over the eight tasks	0.661	0.670	0.738	0.755	0.753	0.750	0.717 [‡]	0.756	0.777

Table 4: Constraint factorial on the frozen LIBERO recipe, powered evaluation (last-3 checkpoints). “Clip off” disables both displacement bounds (δ_Q and δ_{tot}); “anchor off” disables both restoring terms. The $(\times, \times)$ row is the unconstrained action-gradient update — the form DIPO and QSM run (Table 1). The two guards do not compose the same way on the two tasks, and on one of them the step clip actively hurts. On ketchup, full CAST (0.627; seeds 0.68/0.58/0.62) beats anchor-only (0.387; 0.40/0.41/0.35) with no overlap — its worst seed clears the ablation’s best by 0.17. On the red-mug task the ordering reverses and just as cleanly: anchor-only reaches 0.810 (0.85/0.77/0.81) against full CAST’s 0.600 (0.59/0.59/0.62), again without overlap. Adding the step clip to the anchor costs 21 points there. So “both guards are necessary” is not what this factorial shows. What it shows is that the anchor is necessary — the two rows without it are at 0.000 on both tasks — while the clip helps on one task and hurts on the other. Full CAST clears its base on both (+0.027 and +0.017) but only barely, and we no longer claim more than that from this table. “—” = not measured.

clip	anchor	configuration	red mug onto plate (0.573)	ketchup into drawer (0.610)
$\times$	$\times$	unconstrained transport	0.000	0.000
$\checkmark$	$\times$	step containment only	0.000	0.000
$\times$	$\checkmark$	base-restoring only	0.810	0.387
$\checkmark$	$\checkmark$	full CAST	0.600	0.627

Table 5: Dead-zone radius sweep, single seed, powered evaluation (last-3 checkpoints). ρ varies the shape of the restoring field, not its presence. The genuinely unconstrained configuration is Table 4’s first row.

dead-zone radius ρ	red mug onto plate (0.573)	ketchup into drawer (0.610)
0 (always-on penalty)	0.549	0.601
0.02	0.863	0.734
0.05 (recipe)	0.781	0.684
0.10	0.812	0.694
0.20	0.771	0.671
∞ (dead zone inert; weak pull + clip remain)	0.803	0.671

Table 6: Component ablations: the two halves of the update. CAST displaces each candidate by a reward field V_Q and a restoring field V_{anc} . Each block replaces one half and holds the other at CAST’s setting, so the two blocks are different axes, not repeats of one another. Top: what the critic contributes — its gradient, or coarser signals that keep only a ranking. Deleting the gradient direction costs 35–40 points on square and is the larger effect. Bottom: how the excursion is bounded, at matched geometry. The first two rows constrain in the loss or the parameters; the last three act on the action target. What separates the working rows from the failing ones is not where the constraint lives but whether it has a dead zone: the three configurations with a free interior all land in 0.68–0.84, and the two without one (always-on pull, and DICE-RL’s L_2 penalty with gradient clipping) sit at or below base. Our anchor is not the best of the three — see the discussion below. *these two square entries are the same two methods measured in Table 2 at the matched 20K budget (3 seeds), not at the much longer budget the other square locus arms use — they are quoted for reference and should not be compared across columns. The remaining three square cells (hinge, projection, always-on pull) are training. “—” = not measured.

variant	acts on	square (3 seeds)	red mug onto plate (0.573)	ketchup into drawer (0.610)
(a) the reward field V_Q — constraint held at CAST’s				
gradient (CAST)	—	0.912–0.930	0.600	0.627
top- k (ranking only)	—	0.525–0.562	0.577	0.341
tilted ($\exp(Q/\tau)$)	—	0.560–0.584	0.697	0.378
(b) the restoring field — reward held at CAST’s gradient				
L_2 penalty + grad clip (DICE-RL)	loss + params	0.861–0.909*	0.550	0.587
always-on pull ($\rho=0$)	action target	—	0.549	0.601
hinge ($\ r\ - \rho$) $_+^2$	loss	—	0.837	0.700
hard projection $\Pi_{\ r\ \leq \rho}$	action target	—	0.790	0.733
dead-zone anchor (CAST)	action target	0.912–0.930*	0.600	0.627

Table 7: Scope test on the DMC cartpole pair: same distilled base, same critic architecture, dense vs. sparse reward, Scores are undiscounted episode return: the sum of the per-step reward over one episode, averaged over 50 evaluation episodes. DMC gives reward in $[0, 1]$ at every step and runs a fixed 1000 steps with no early termination, so the scale is 0–1000 by construction and the “normalized” column is simply return/1000. Three reference points make the numbers readable, all measured from our own data rather than quoted: the demonstrations average 429 (which is what the BC base scores, 0.43 normalized), the SAC teacher that generated them ended at 633, and the single best demonstration episode was 729. The last is the meaningful bar — an arm that clears it has produced behaviour better than anything in its training data — and only CAST clears it, on one of two seeds. Cells give both seeds (42 / 43); all twelve runs completed. The dense column is the informative one and it is consistent across seeds: the unconstrained update not only survives dense reward but ends well above the base (≈ 430), which is the opposite of its behaviour under sparse manipulation reward, where it reliably reaches 0.000. The sparse column here is not interpretable — the unconstrained arm returns 57 on one seed and 514 on the other — so we draw no conclusion from it and report it only for completeness. What the dense column supports is the boundary claim: sparsity is not what causes the policy to leave the data, it is what removes any route back.

actor update	swingup (dense)			swingup_sparse
	return (s42 / s43)	normalized	clears best demo?	return (s42 / s43)
reference: base = 429		0.43		
unconstrained transport	508 / 642	0.51 / 0.64	no	57 / 514
backprop $-Q$ (DICE-RL form)	294 / 343	0.29 / 0.34	no	4 / 26
CAST	802 / 628	0.80 / 0.63	s42 only	23 / 12

Table 8: Recipe sensitivity: leave-one-out on the frozen LIBERO recipe, powered evaluation (last-3 checkpoints). Each row changes exactly one component; Δ is relative to the full recipe. Resolution-dial rows are in Table 6. “—” = not measured.

variant	the red-mug task (base 0.573)		the ketchup task (base 0.610)	
	success	Δ	success	Δ
full recipe	0.781	—	0.684	—
– dimension-independent anchor (vector norm)	0.662	−0.119	0.655	−0.029
– dead zone ($\rho=0$)	0.549	−0.232	0.601	−0.083
– doubled reward step ($\eta_Q=0.5$)	0.769	−0.012	0.615	−0.069
– 3-step returns ($n=1$)	0.782	+0.001	0.512	−0.172
wider trust region ($\delta_{\text{tot}}=0.25$)	0.874	+0.093	0.689	+0.005
stronger reward step ($\eta_Q=2.0$)	0.802	+0.021	0.729	+0.045

Table 9: Reward-step sweep, powered evaluation (last-3 checkpoints, single training seed). The optimum is task-dependent and non-monotonic: $\eta_Q=4$ is best on the red-mug task, $\eta_Q=1$ on ketchup, and $\eta_Q=8$ is worst on both. $\eta_Q=2$ is best on neither individually but has the best mean across the eight tasks of Table 3, which is why it is the setting we ship. On the flow-matching base the ordering differs again — $\eta_Q=2$ beats $\eta_Q=1$ on all three tasks measured — so the shared setting is a compromise across bases as well as tasks. “—” = not run.

base	task (own base)	$\eta_Q=1$	$\eta_Q=2$	$\eta_Q=4$	$\eta_Q=8$
drifting	red mug onto plate (0.573)	0.775	0.802	0.880	0.768
drifting	ketchup into drawer (0.610)	0.737	0.720	0.721	0.686
flow matching	ketchup into drawer (0.690)	0.738	0.773	—	—
flow matching	red mug onto plate (0.787)	0.878	0.887	—	—
flow matching	book into front caddy (0.683)	0.667	0.783	—	—

Table 10: Shared-critic action-sensitivity on the LIBERO hard-8 multitask policy, measured on $B=8$ states $\times$ $K=16$ candidates. The two rows are the extremes across tasks; the diagnostic labels the first “alive” and the second “DEAD (flat critic)”. Even where the gradient has magnitude, a 0.05 rms action step changes Q by under 1 part in 40 of $|Q|$, and the 16 candidates are scored within 0.7% of one another.

	gradient “alive”	gradient “dead”
$\ \nabla_a Q\ $ (per-particle mean)	5.49×10^{-1}	3.10×10^{-3}
$ \Delta Q $ for a 0.05 rms action step	9.21×10^{-4}	7.05×10^{-5}
$ Q $ mean	0.043	0.579
$\text{std}(Q)/ Q $ across $K=16$	0.0418	0.0003
$Q_{\max} - Q_{\min}$ across $K=16$	6.94×10^{-3}	7.85×10^{-4}

Table 11: Scale of QSM’s score-matching target against the denoiser’s own output, square base, critic at initialisation.

η (q_grad_coeff)	1	10 (published)	100
$\ \eta \nabla_a Q\ $	0.013	0.127	1.266
ratio to $\ \epsilon\ $	0.0024	0.0243	0.2428

Table 12: Distance from the frozen base at end of training, $D_{\text{base}} = \mathbb{E}\|r_\theta\|_{\text{rms}}$, red-mug task, $\rho = 0.05$.

configuration	D_{base}	powered success
no constraint	7.92	0.000
step clip only	21.70	0.000
dead-zone anchor only	0.050	0.810
hinge in the loss	0.048	0.837
full CAST	0.016	0.600

Table 13: The bound on H2: where the critic has no usable signal, no update rule helps — including ours. LIBERO hard-8 with a single policy and a single critic trained on all eight tasks jointly, in contrast to the per-task routing of Table 3. Powered evaluation (100 rollouts $\times$ 3 evaluation seeds; one column per training seed; 100 RL iterations). This is a negative result and we report it as one: read the Δ column, where nothing moves its own base by more than seed noise (± 0.014). Two rows use the backpropagated actor and they are not duplicates — “its published recipe” is DICE-RL as shipped ($\beta_{BC}=100$, best-of- N exploration), while “our recipe” is the same actor under the settings CAST uses ($\beta_{BC}=10$), so it is the matched control. Δ is against each row’s own base, the only fair comparison across the two blocks: the flow-matching numbers are larger in absolute terms purely because that base starts higher. The final block is an ablation of one component — how the critic scores the K candidates before displacement — and is not a separate method; the corresponding single-task sweep is Table 6.

method	s10000	s10001	s10002	Δ vs. own base
drifting base				
drifting base (no RL)	0.671 ± 0.014			—
drifting + DICE-RL (its published recipe)	0.705	0.690	0.670	+0.017
drifting + backprop actor (our recipe)	0.657	0.655	0.668	−0.011
drifting + DICE-RL + CAST (our recipe)	0.655	0.675	0.659	−0.008
flow-matching base				
flow-matching base (no RL)	0.738 ± 0.022			—
flow matching + DICE-RL	0.756	0.735	0.737	+0.005
flow matching + DICE-RL + CAST	0.746	0.729	0.746	+0.002
ablation: how CAST scores candidates (drifting base)				
top- k scoring (mean Q of best k)	0.665	0.684	0.668	+0.001
tilted scoring (Q -weighted average)	0.677	0.627	0.672	−0.013